



分布式计算

余晨韵

中山大学智能工程学院

第三章：分布式协调与同步

参考：1、分布式系统概念和设计，第14,15章
2、分布式系统原理和范型，第6章

第三章：分布式协调与同步

- 3.1 时钟与同步: Lamport算法、向量时钟、矩阵时钟
- 3.2 分布式互斥: 集中式算法、分布式协商算法、令牌环算法
- 3.3 分布式选举: Bully算法、Raft选举算法、ZAB选举算法
- 3.4 分布式共识: PoW、PoS、DPoS
- 3.5 总结

3.1 时间同步

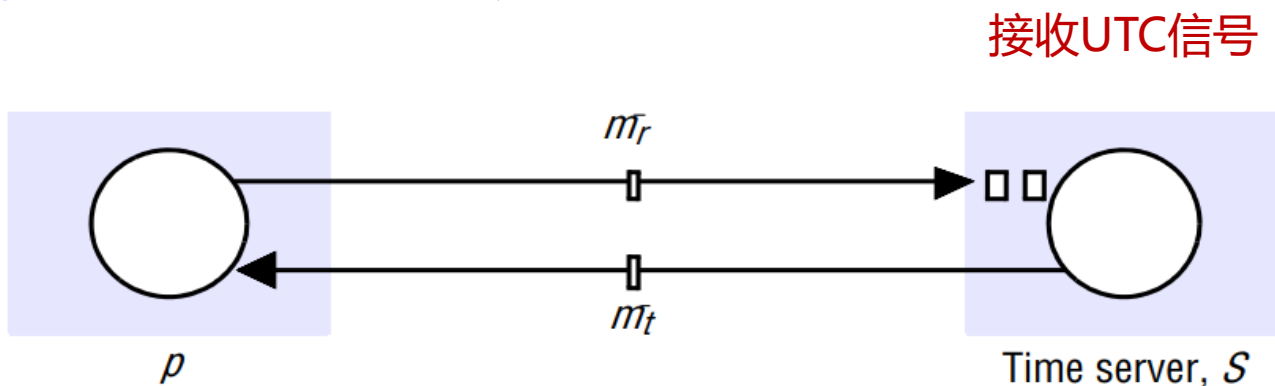
◆ 背景介绍

- 分布式系统需要比较不同节点上时间的发生顺序
- 不同计算机的物理时钟存在偏差
 - ✓ 受时钟分辨率（时钟值更新的周期）影响
 - ✓ 存在时钟偏移和时钟漂移（晶体震荡周期不同）
- 物理时钟同步是一种解决方式，但是误差依旧存在。
- 通用协调时间（UTC）：基于原子时间，是国际计时标准，由广播电台和GPS同步和广播。



3.1 时间同步

◆ 同步物理时钟：Cristian方法



- 进程 P 发送消息 m_r 到服务器 S 请求时钟； S 将当前时间 t 放入消息 m_t 中发给 P ，整个消息往返过程的时间为 T_{round}
- 一般情况下， P 应该将它的系统时间设置为 $t + T_{\text{round}}/2$
- 若 $\min$ 为 P 与 S 的最小传输时间，那么应答消息到达 P 的时间应当介于 $[t + \min, t + T_{\text{round}} - \min]$

3.1 时间同步

◆ 同步物理时钟：Berkeley方法（内部同步）

- 选择一台协调者计算机作为主机（master），主机定期轮询其他要同步时钟的计算机（从属机slave）；
- 主机观察消息的往返时间来估计从属机的本地时间，并计算所获值的平均数；
- 主机采用容错平均值：在时钟中选择差值不多于某个阈值的子集，再计算平均值；
- 主机通知每个从属机时钟所需要的调整量；
- 主机如果出现故障，则选举另一个主机接管。

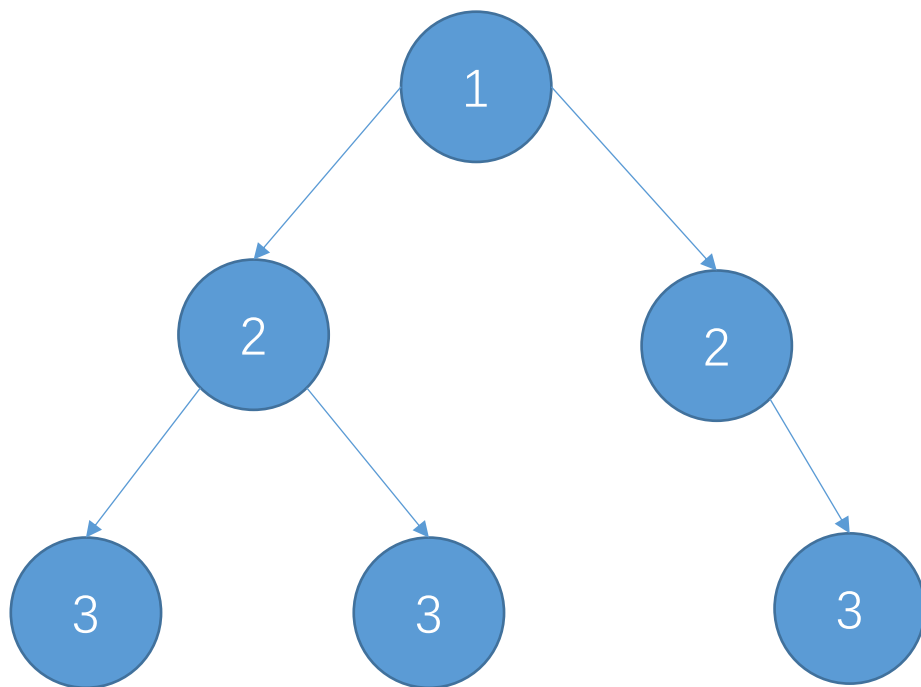
3.1 时间同步

◆ 网络时间协议NTP (Network Time Protocol)

- ▣ 提供一个服务，使得跨互联网的用户能精确地与UTC同步
- ▣ 提供一个能在漫长的链接丢失中生存的可靠服务：冗余服务器
- ▣ 使客户能有效地重新同步以抵消在大多数计算机中存在的漂移率
- ▣ 采用认证技术，防止对时间服务的干扰（恶意/偶然）

3.1 时间同步

◆ 在NTP中实现同步子网的例子



- 多层服务器架构
- 主服务器接受UTC时间源
- 层次2服务器与主服务器直接同步
- 层次3服务器与层次2服务器同步
- 缺点：层次越多，偏差越大

3.1 时间同步

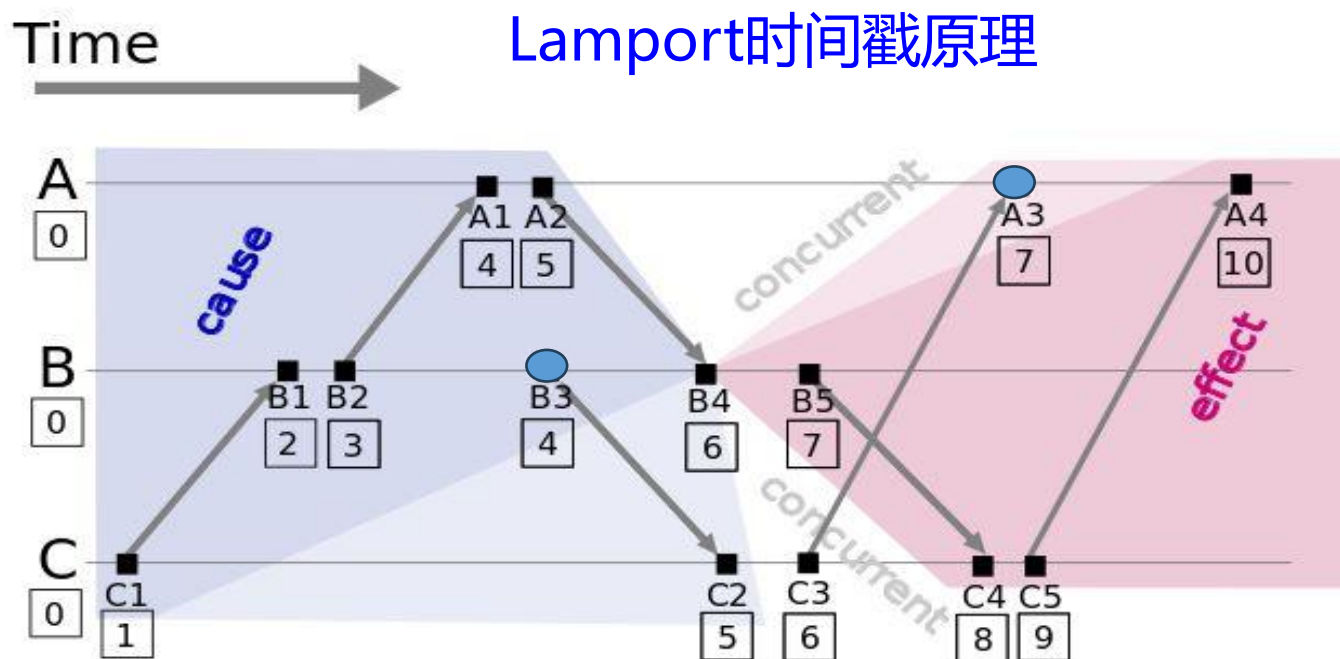
◆ 逻辑时钟

- **定义：** 分布式系统中用于记录事件顺序关系，是一个单调增长的计数器
- [Leslie Lamport](#) 提出 Lamport时间戳作为逻辑时钟的表示方法

◆ 时间的顺序

- 如果两个事件发生在同一个进程P，那么发生顺序为进程P观察到的顺序；
- 当消息在不同进程中发生时，发送消息的事件 **早于** 接收消息的事件

3.1 时间同步

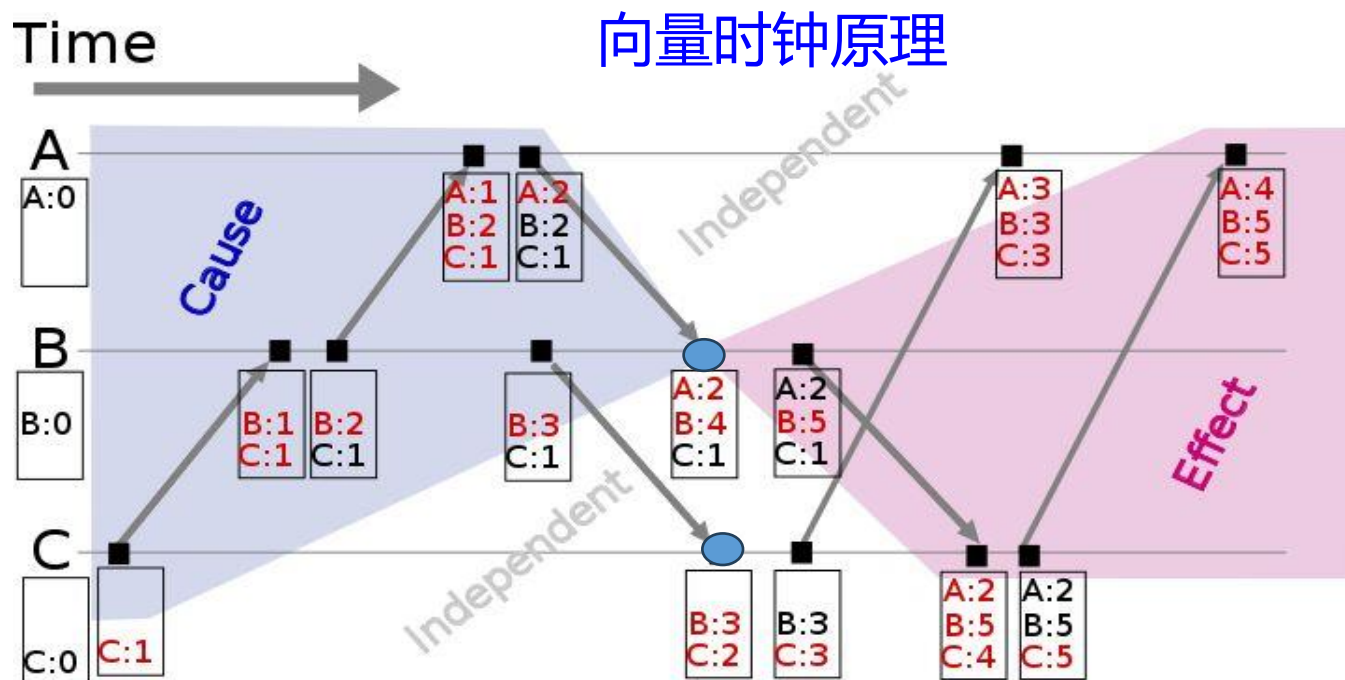


- 每个事件对应一个Lamport时间戳，初始值为0
- 如果事件在节点内发生，本地时间戳加1

若时间戳相同，
事件先后顺序难以判断

- ✓ 如果事件属于发送事件，在消息中带上该时间戳
- ✓ 如果事件属于接收事件，时间戳 = $\text{Max}(\text{本地时间戳}, \text{消息中的时间戳}) + 1$

3.1 时间同步



- Lamport时间戳帮助我们得到事件顺序关系
- 向量时钟能衡量同时发生的事件如何表示先后顺序
- 通过vector结构不但记录本节点的时间戳，同时也记录了其他节点的时间戳
- 缺陷：占用更多存储空间；若两个事件无因果联系，可能存在歧义，难以判断

3.1 时间同步

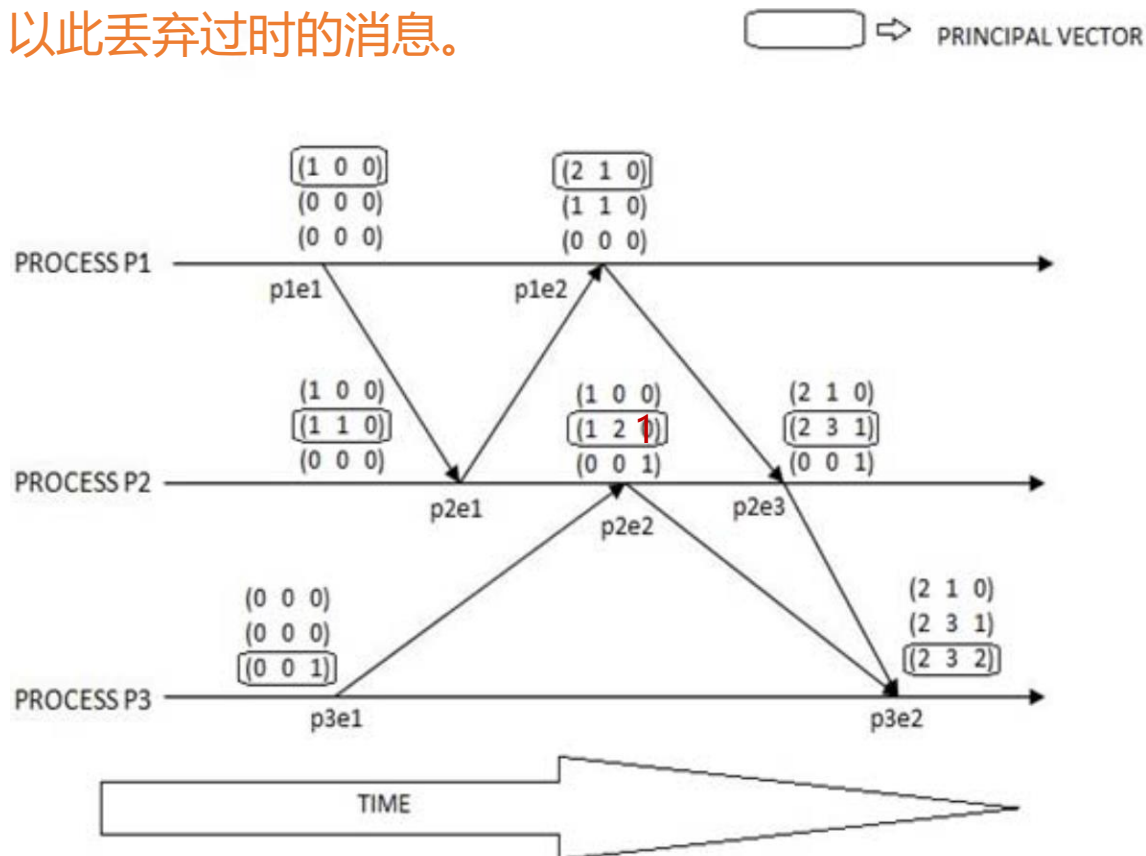
◆ 矩阵时钟

- 矩阵时钟在向量时间的基础上，添加了系统其他进程视图的信息。
 - 每个进程 p_i 维护一个 $n \times n$ 的非负整数矩阵 mt_i ，表示 p_i 逻辑全局时间的局部视图。 $mt_i[i,:]$ 相当于向量时钟 $v[i]$ 。
 - 当有新事件发生时， $mt_i[i,i] = mt_i[i,i] + d$ ($d > 0$)。
 - 当进程 p_i 发送消息时将它的矩阵时间戳附带在消息(m, mt)中。
 - 当 p_i 收到 p_j 发送的消息 (m, mt) 时，按最大时间戳更新本地时间：
 - ✓ $1 \leq k \leq n : mt_i[i,k] = \max(mt_i[i,k], mt[j,k])$
 - ✓ $1 \leq k, 1 \leq n : mt_i[k,l] = \max(mt_i[k,l], mt[k,l])$
 - ✓ $mt_i[i,i] = mt_i[i,i] + d$ ($d > 0$)

3.1 时间同步

◆ 矩阵时钟 （更新=自身状态更新+接收到的消息）

- 矩阵时钟除了包含向量时钟的所有特性之外，还有 $\min(mti[k,i]) \geq t$ ，即 p_i 知道进程 p_k 对 p_i 的时间了解已经到达 t 。其他进程不需要 p_i 发送 t 之前的消息，以此丢弃过时的消息。



第三章：分布式协调与同步

- 3.1 时钟与同步
- 3.2 分布式互斥
- 3.3 分布式选举
- 3.4 分布式共识
- 3.5 总结

3.2 分布式互斥

- ◆ 什么是分布式互斥?
- ◆ 为何需要分布式互斥?
- ◆ 分布式互斥方法介绍
- ◆ 小结与思考

3.2 分布式互斥

◆ 什么是分布式互斥？

- 定义：分布式系统里排他式的资源访问方式
- 例子：使用自助咖啡机泡咖啡



◆ 临界资源：被互斥访问的共享资源

- 写数据到某个指定文件

◆ 如何让分布式系统的程序互斥地访问临界资源？

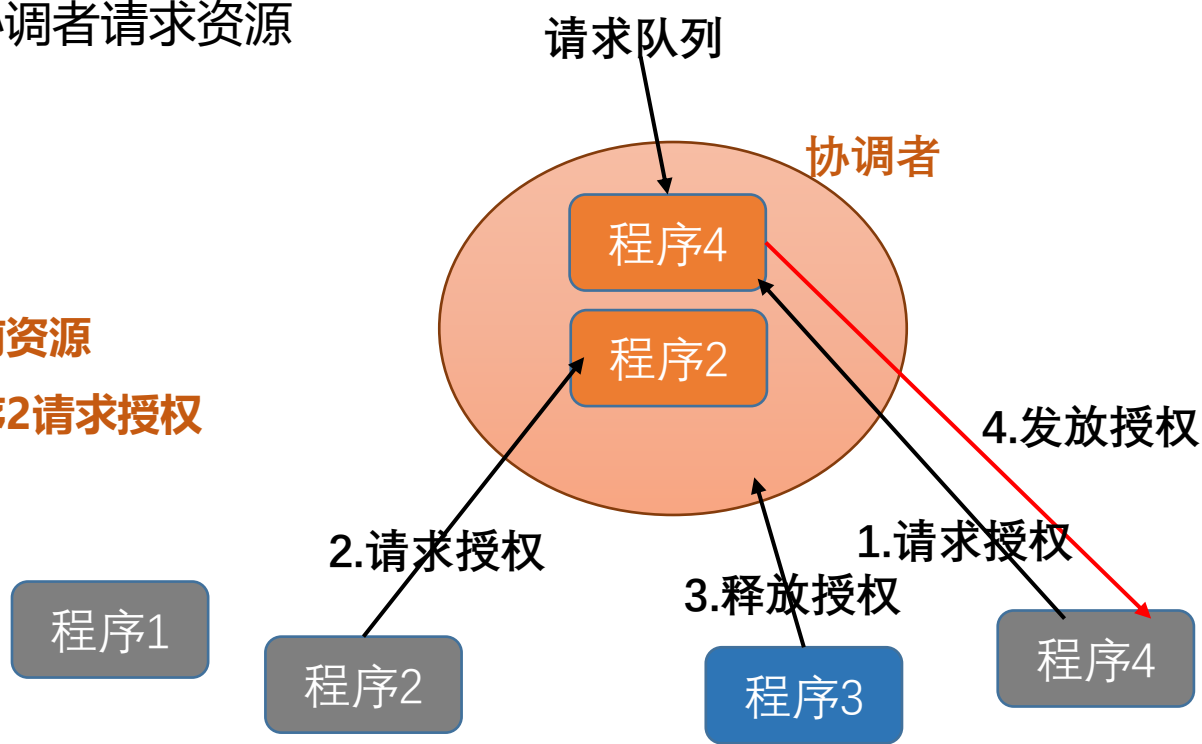
3.2 分布式互斥：算法

◆ 方案一：集中式算法

- 引入一个协调者程序（中央服务器）
- 使用者需要向协调者请求资源

■ 当前状态：

- 程序3占用当前资源
- 程序4早于程序2请求授权



3.2 分布式互斥：算法

◆ 方案一：集中式算法

- ① 向协调者发送请求信息，1次消息交互
- ② 协调者向程序发放授权信息，1次消息交互
- ③ 程序使用完临界资源后，向协调者发送释放授权，1次消息交互

◆ 优点：直观简单、信息交互少、易于实现、只需与协调者通信

◆ 缺点：协调者成为性能瓶颈，易引发单点故障

□ 100个程序访问资源，消息数=300

◆ 适用条件：选择性能好，可靠性高的服务器运行协调者

□ 设置备用服务器，且数据可恢复

□ 授权相当于“分配锁”

3.2 分布式互斥：算法

◆ 方案二：民主协商（分布式算法）

□ 思路：去除协调者，解决单点故障

✓ 例子：咖啡机使用

□ 征求其他参与者同意，再使用资源

✓ 请求消息（请求者ID，请求的资源，时间戳）

✓ 发消息：组播方式

✓ 同步：逻辑时钟

□ 先到先得，投票全通过原则



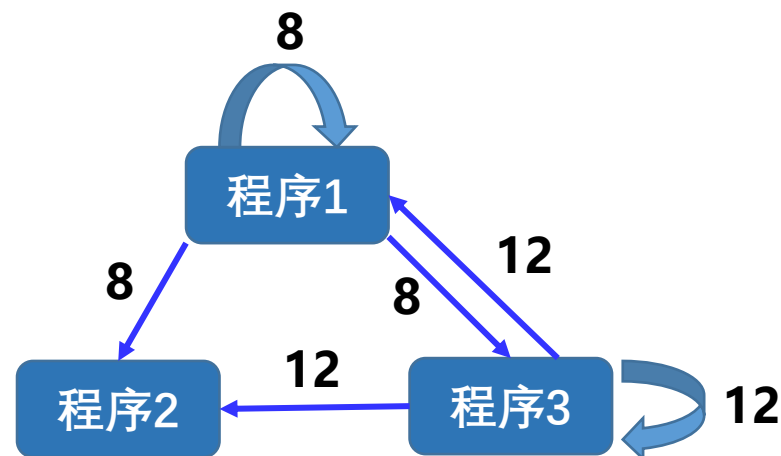
3.2 分布式互斥：算法

◆ 方案二：民主协商（分布式算法）

- ① 时间戳=8时，程序1想使用资源A，向其他用户发起请求
- ② 时间戳=12，程序3想使用资源A，向其他用户发起请求

程序1维护的请求队列（当时间戳>12）

程序3，资源A，时间戳12



程序1和程序3差不多同一时间要访问共享资源A

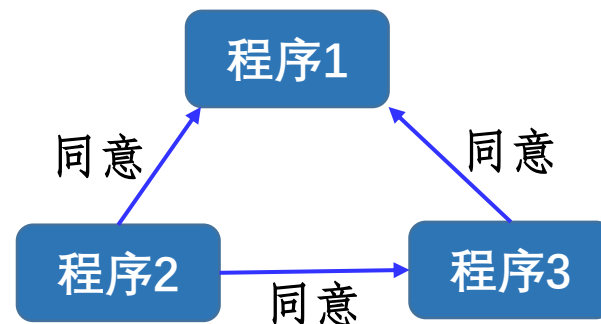
3.2 分布式互斥：算法

◆ 方案二：民主协商（分布式算法）

- ③ 程序2暂不访问资源A，因此同意程序1与程序3的请求
- ④ 程序1时间早于程序3，因此程序3同意程序1使用A

程序1维护的请求队列

程序3，资源A，时间戳12



程序1的请求时间早于程序3，获得授权，访问资源A

3.2 分布式互斥：算法

◆ 方案二：民主协商（分布式算法）

- ⑤ 程序1使用完资源A后，释放使用权限
- ⑥ 程序1检查请求队列，向程序3发送同意消息
- ⑦ 程序1将程序3从请求队列中清除

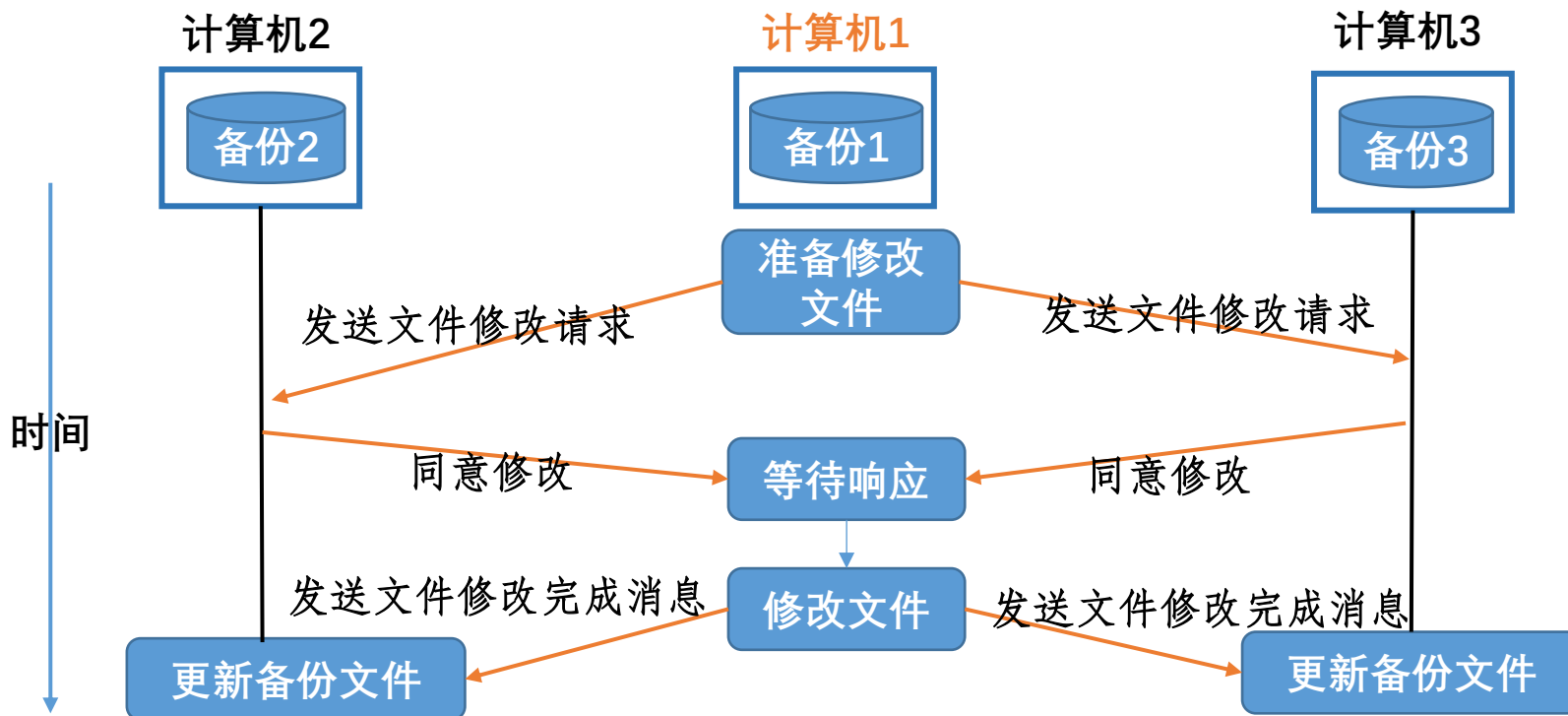


程序1释放资源A，程序3获得所有授权，访问资源A

3.2 分布式互斥：算法

◆ 应用场景：分布式文件系统HDFS的文件追加写内容

- 计算机1,2,3有同一份文件的备份，计算机1想要修改共享的文件



3.2 分布式互斥：算法

◆ 方案二：民主协商（分布式算法）

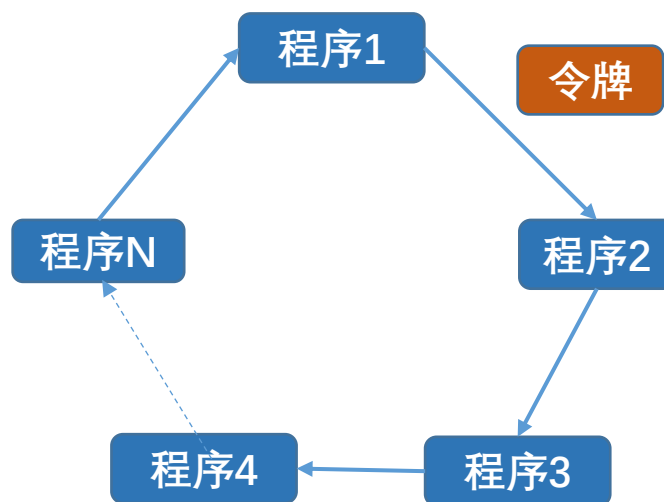
- 优点：程序按“先来后到”公平地访问资源，简单粗暴易实现
- 缺点1：容易产生“信令风暴”
 - ✓ N 个程序均访问临界资源，会产生 $2N(N-1)$ 条消息
- 缺点2：可用性低
 - ✓ 一旦发生故障，系统容易进入等待或停滞
 - ✓ 改进：检测到故障程序，则忽略（超时设置）
- 适合临界资源使用频率较低且系统规模较小的场景



3.2 分布式互斥：算法

◆ 方案三：轮值CEO（令牌环算法）

- 多个参与者组成环，依次传递令牌
- 持有令牌方可使用资源
- 若不使用，则把令牌传给下一个程序



令牌环算法示意图

◆ 令牌环算法的优点：

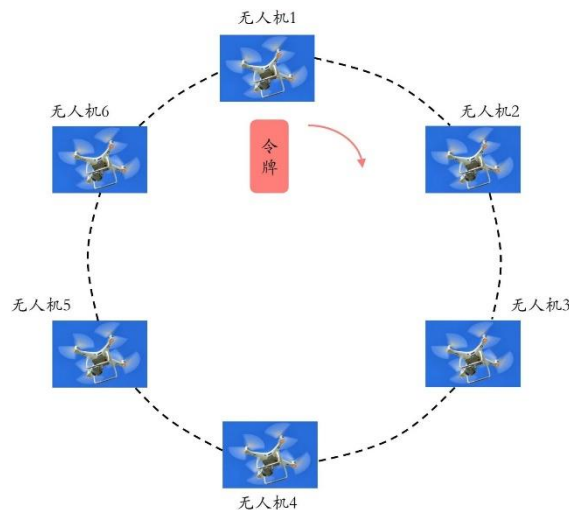
- 无须征求其他程序的意见，比分布式算法的通信效率高
- 一个周期内，每个程序都能访问到临界资源，算法公平性好

3.2 分布式互斥：算法

◆ 令牌环算法的缺点：

□ 无论是否想要访问资源，都要接收并传递令牌环，会带来无效的通信

✓ 例如：系统中100个程序只有1个需要访问资源，且需要多个令牌环时间才能执行完毕，等候时间长。



无人机通信示意图

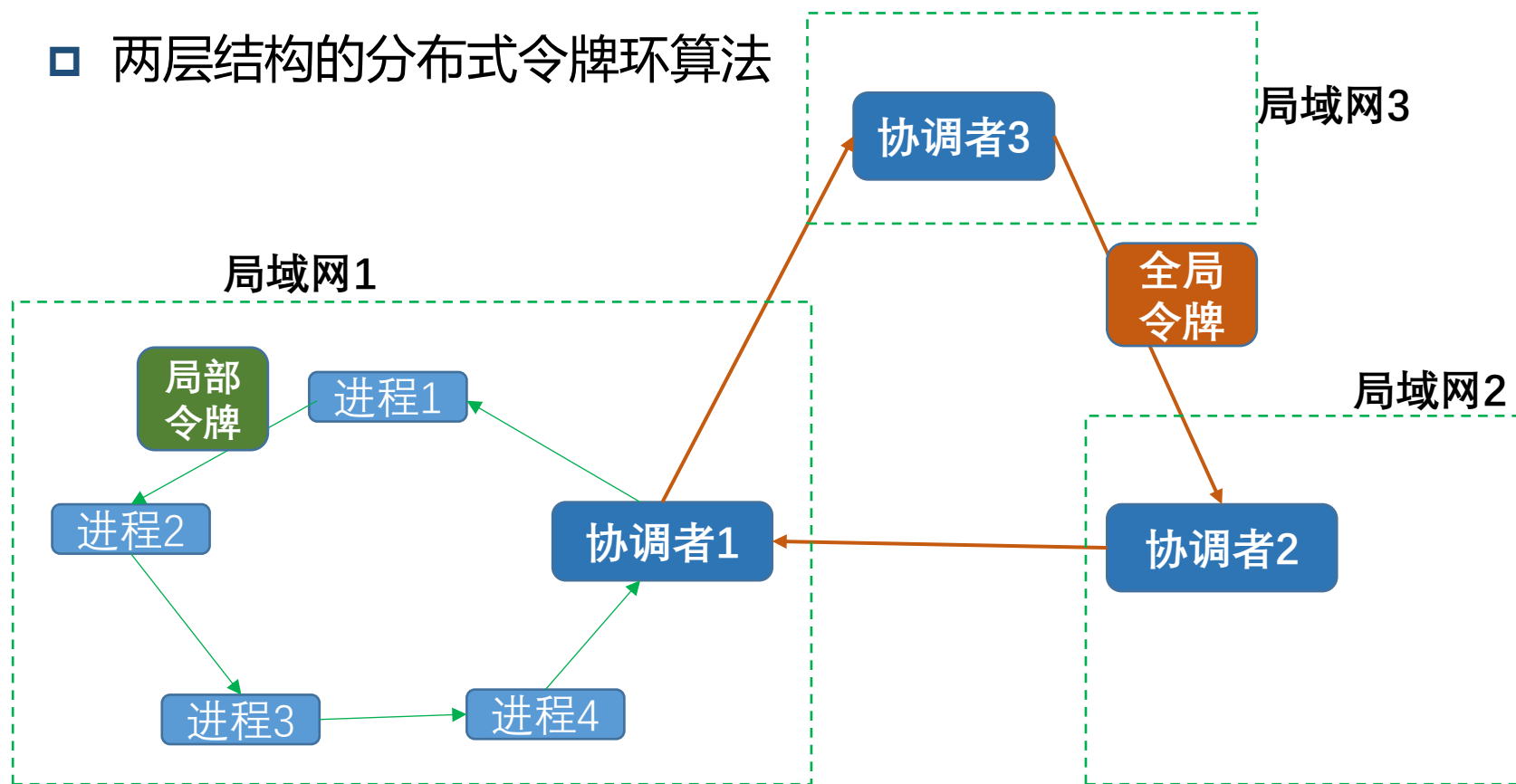
◆ 令牌环算法的适用场景：

□ 系统规模小，每个程序使用资源频率高且时间短，例如无人机组通信。

3.2 分布式互斥：算法

◆ 知识拓展：有适合大规模系统中的分布式互斥算法么？

□ 两层结构的分布式令牌环算法



分布式互斥方法

霸道总裁：集中式方法

方法

引入一个协调者，将所有的请求者进行排序，最早请求的参与者可以使用临界资源

优点

简单、易于实现

通信高效

缺点

可用性低

性能易受协调者影响

应用场景

在协调者可靠性和性能有一定保障的情况下，可以适用于比较广泛的场景

民主协商：分布式方法

方法

征求其他参与者同意后，使用临界资源

优点

可用性较高

缺点

通信成本较高

复杂度较高

应用场景

临界资源使用频度较低且系统规模较小的场景

轮值CEO：令牌环方法

方法

所有参与者组成一个环，轮流使用资源

优点

单个参与者通信效率较高

可用性较高

缺点

当参与者对临界资源使用频率较低时，会带来较多无用通信

应用场景

系统规模较小，并且系统中每个程序使用共享资源频度较高且使用时间较短的场景

3.2 分布式互斥: 思考

- ◆ 你认为集中式算法、分布式算法和令牌环算法，还有什么可以改进的地方？
 - 集中式：考虑用集群提高协调者的可靠性，涉及复制，选主，类似zookeeper集中式锁
 - 分布式算法：设置等待时间，采用大多数投票
 - 令牌环：根据使用频率调整权重，平滑加权轮询

第三章：分布式协调与同步

- 3.1 时钟与同步
- 3.2 分布式互斥
- 3.3 分布式选举
- 3.4 分布式共识
- 3.5 总结

3.3 分布式选举

- ◆ 为何需要分布式选举?
- ◆ 分布式选举方法介绍
 - Bully算法、Raft选举、ZAB算法
- ◆ 小结与思考

3.3 分布式选举

◆ 为何需要分布式选举？（国不可一日无君）

- 许多分布式算法需要选出一个进程充当协调者、发起者、排序者等
 - ✓ 例如：数据库集群需要保证写入的数据在每个节点上都一致
- 如果协调者发生故障，则必须选择另一个副本代替其运行

◆ 集群中如何选举？

- 定义每个节点的优先级作为选举参考条件
- 在所有节点都同意的基础上选出优先级最大的节点作为协调者

◆ 常用算法：

- 基于序号（优先级）的选举: Bully算法
- 多数派算法（例如Raft算法，ZAB算法）

3.3 分布式选举：Bully算法

◆ Bully 算法：长者为大

- 假设条件：集群中每个节点都知道其它节点的ID（ID 大代表优先级高）
- 霸道的集群选主算法，在所有活着的节点中，选ID最大的为主节点
- 当且仅当主节点故障或与其他节点失去联系时，才会重新选主
- 选举过程需要用到三种消息：
 - ✓ Election消息：用于发起选举
 - ✓ Alive消息：若优先级高于对方，则对Election消息进行应答
 - ✓ Victory消息：竞选成功的主节点向其它节点发送的宣誓主权的消息

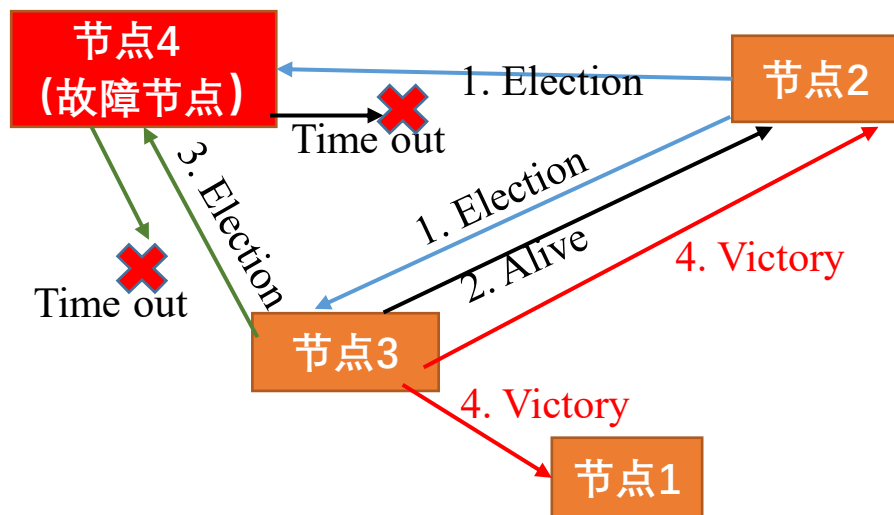
3.3 分布式选举：Bully算法

Bully 算法选举过程：

- ① 集群中每个节点判断自己的ID是否为当前最大
 - Yes, 则直接向其他节点发送Victory消息, 宣誓自己的主权;
 - No, 向比自己ID大的所有节点发送Election消息, 并等待其他节点的回复;
 - 时间T内, 本节点没有收到其他节点回复的Alive消息, 则认为自己成为主节点, 并向其它节点发送Victory消息;
 - 若收到来自比自己ID大的节点的Alive消息, 则等待其他节点发送Victory消息;
 - ② 若本节点收到比自己ID小的节点发送的Election消息, 则回复Alive消息, 告知其他节点, 本节点重新选举。
 - ③ 当一个ID更大的节点因故障排除恢复运行时, 它重新执行竞争者算法, 以剥夺较小ID节点的协调者资格, 自己重新行使此权利。
-

3.3 分布式选举：Bully算法

◆ Bully 算法：实例



- ① 节点2向3、4发送Election消息
- ② 节点3回复Alive，节点2进入等待
- ③ 节点3向节点4发送Election消息
- ④ 节点4无回复，节点3胜利
- ⑤ 节点3给1、2发送Victory

- 优点：选举速度快、易理解、算法复杂度低、易于实现
- 缺点：存储所有节点的ID、易频繁切主、不适用于大规模分布式系统

3.3 分布式选举：Raft

◆ 民主投票：Raft算法

- 核心思想：少数服从多数，获票数最多成为主节点
- 集群节点的角色有3种：
 - Leader：主节点，负责协调和管理其他节点
 - Candidate：即候选者，每个节点都可以成为Candidate，只有在该角色下才可以被选为Leader
 - Follower：Leader的跟随者，不可发起选举

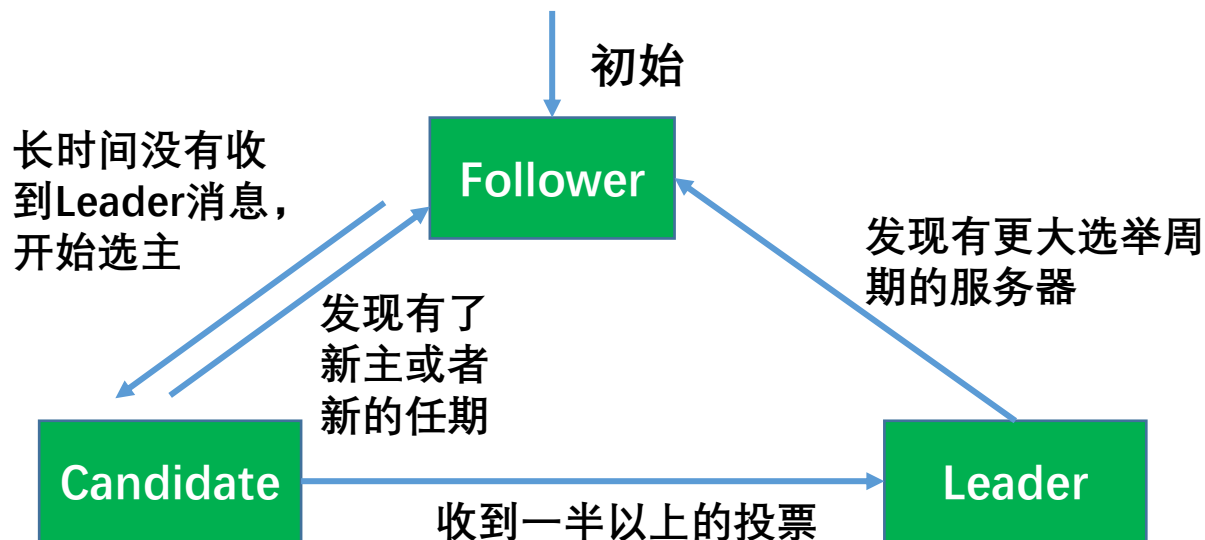
3.3 分布式选举： Raft

◆ Raft选举的流程：

1. 初始化时，所有节点均为Follower状态；
2. 开始选主时，所有节点状态转为Candidate，并向其它节点发送选举请求；
3. 其它节点根据收到的选举请求先后顺序，回复是否同意其成为主。（注意，每轮选举中，一个节点只能投出一张票）
4. 若发起选举请求的节点获得超过一半的票数，则成为主节点，状态转为Leader，其它节点的状态则变为Follower；
5. 当Leader节点的任期到了，且发现其它服务器开始下一轮选主周期时，Leader节点的状态由Leader降为Follower，进入新一轮选主

3.3 分布式选举： Raft

◆ Raft选举的节点状态转移如下图：

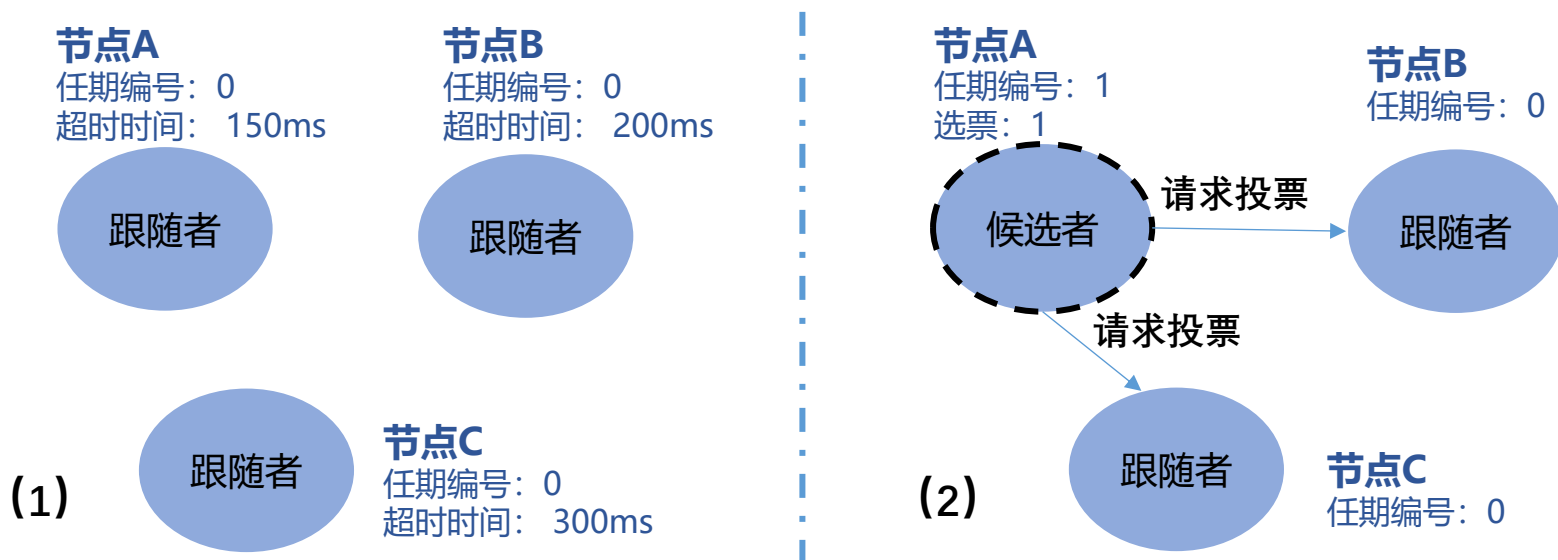


- 每一轮选举，每个节点只能投一次票
- 选主时周期进行，包括选主和任期，如果主节点故障，立马发起新的选举

3.3 分布式选举： Raft

◆ 选举领导者的过程：

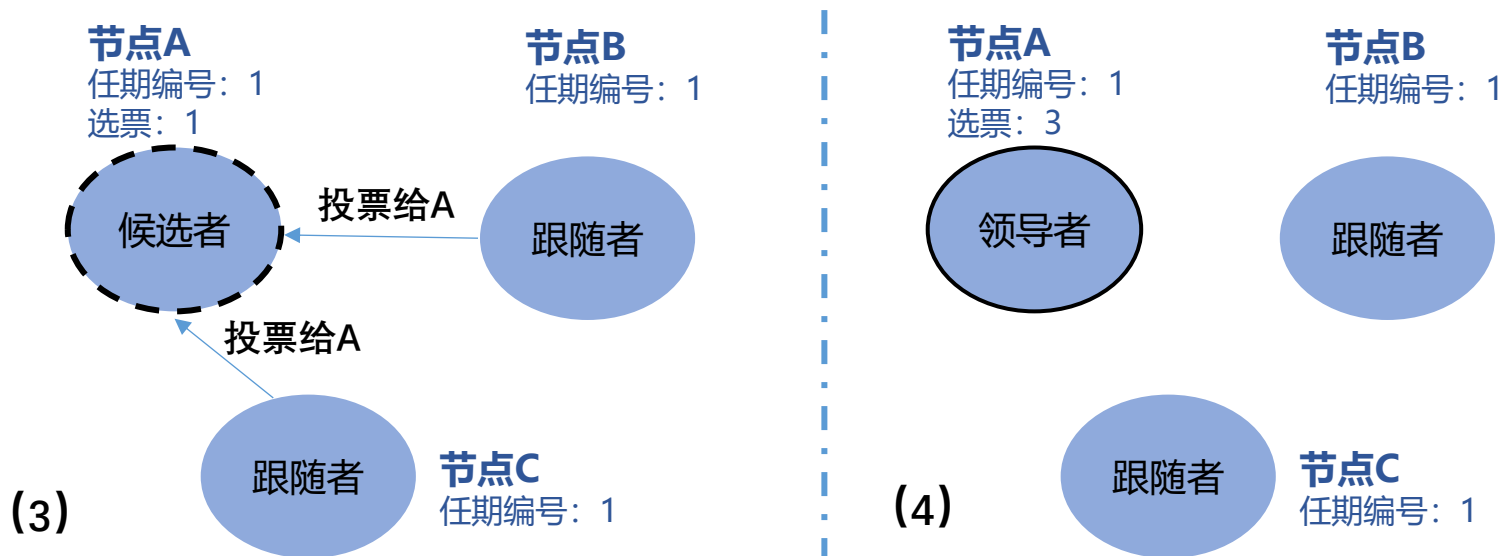
- ① 首先，在初始状态下，集群中所有的节点都是跟随者状态。
 - ✓ 每个节点等待领导者心跳信息的超时时间间隔是随机的；
 - ✓ 此时没有领导者，节点A的等待超时时间最小 (150ms), 第一位发生超时；
- ② 节点A增加自己的任期号，推举自己为候选人并给自己投1票，然后向其他节点发送请求投票RPC消息，请他们选自己为领导者。



3.3 分布式选举：Raft

◆ 选举领导者的过程：

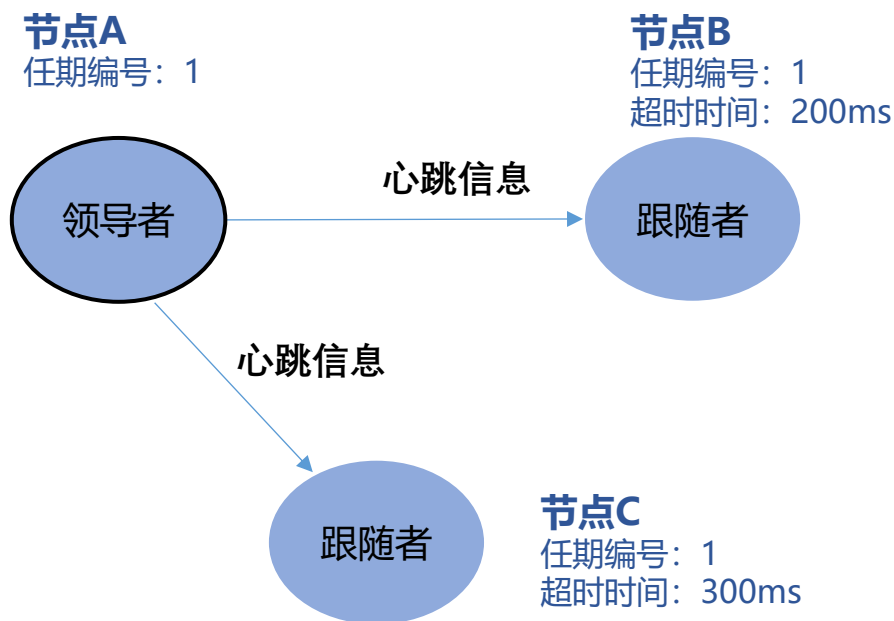
- ③ 若其他节点收到候选人A的请求投票RPC消息，且在编号为1的这届任期内还没投过票，那么它将把票投给A，并增加自己的任期编号。
- ④ 如果候选人在选举超时时间内赢得了大多数的选票，那么它就会成为本届任期内新的领导者。



3.3 分布式选举： Raft

◆ 选举领导者的过程：

- ⑤ 节点A当选领导者后，它将周期性地发送心跳消息，通知其它服务器我是领导者，阻止跟随者发起新的选举，篡权。



3.3 分布式选举： Raft

◆ Raft选举：什么是任期？

- 领导者任命到期后要重新开始选举，任期是递增的，充当逻辑时钟的角色。
- 任期编号会随着选举的进行而变化：
 - a. 跟随者在等待领导者心跳信息超时后会推荐自己为候选人，然后增加任期编号；
 - b. 如果一个服务器节点，发现自己的任期比别人小，它会更新到较大的编号值。
- 若候选人/领导者发现自己的任期编号比其他节点小，那么它会立即恢复为跟随者状态。
 - ✓ 例如：网络分区错误恢复后，任期为3的领导者B收到了来自新领导C（任期编号4）的心跳信息，那么B转为跟随者。
- 若一个节点接收到一个包含较小任期编号值的请求，那么它会直接拒绝这个请求。

3.3 分布式选举： Raft

◆ 选举有哪些规则？

- ① 领导者周期性向所有跟随者发送心跳消息，阻止跟随者发起新的选举；
- ② 若跟随者在指定时间内没有收到自领导者的心跳，则推荐自己为候选人，发起新的选举；
- ③ 在一次选举中，赢得大多数选票的候选人将晋升为领导者；
- ④ 一个任期内，领导者将一直工作，直到出现故障（宕机），或者网络延迟，其他节点发起新一轮选举；
- ⑤ 选举遵循“一人一票，先到先得”的投票原则；

3.3 分布式选举： ZAB

ZAB (ZooKeeper Atomic Broadcast) 选举算法

◆ **核心思想**：具有优先级的民主投票，尽可能保证数据的最新性

- 节点ID代表等级大小(能力强弱程度) ， 数据ID代表新旧程度
- 节点ID和数据ID越大，优先成为主

◆ **集群中每个节点有3种角色**：

- Leader：主节点；
- Follower：跟随者节点；
- Observer：观察者，无投票权

3.3 分布式选举： ZAB

◆ 每个Server在工作过程中有四种状态：

- ▣ Looking：当前Server不知道leader是谁，正在搜寻
- ▣ Leading：当前Server即为选举出来的leader
- ▣ Following：leader已经选举出来，当前Server与之同步
- ▣ Observing：节点跟随 Leader 保存系统最新的状态提供读服务，但不参与选举和事务投票

3.3 分布式选举： ZAB

◆ 选票的数据结构：

- 每个服务器在进行leader选举时，会发送以下信息（选票）：

变量	描述
logicClock	代表这是该服务器发起的第几轮投票
state	当前服务器的状态
Self_id	当前服务器的myid
Self_zxid	当前服务器上所保存的数据的最大zxid
Vote_id	被推举的服务器的myid
Vote_zxid	被推举的服务器的服务器上所保存的数据的最大zxid

3.3 分布式选举： ZAB

◆ 投票流程：

① 自增选举轮次

- ▣ 每个服务器在开始新一轮投票时，会先对自己的 logicClock 进行自增操作。

② 初始化选票

- ▣ 每个服务器在广播自己的选票前，会将自己的投票箱清空。

③ 发送初始化选票

- ▣ 给自己投一票，并把投票结果广播出去

④ 接收外部投票

- ▣ 服务器从其它服务器获取投票，并记入自己的投票箱内。如果无法获取任何外部投票，则会确认自己是否与集群中其它服务器保持着有效连接。如果是，则再次发送自己的投票；如果否，则马上与之建立连接。

3.3 分布式选举： ZAB

◆ 投票流程：

⑤ 判断选举轮次，根据选票中的logicClock进行不同处理：

- ▣ 外部投票的logicClock大于自己的logicClock，则立即清空投票箱并将logicClock更新为收到的logicClock，然后再对比自己之前的投票与收到的投票以确定是否需要变更自己的投票，最终再次将自己的投票广播出去；
- ▣ 外部投票的logicClock小于自己的logicClock。当前服务器直接忽略该投票；
- ▣ 外部投票的logicClock与自己的相等，则进行选票PK。

3.3 分布式选举： ZAB

◆ 投票流程：

⑥ 选票PK (选票PK是基于(self_id, self_zxid)与(vote_id, vote_zxid)的对比)

- 先比较二者的 zxid, 若外部投票的 vote_zxid 较大, 则将 (self_id , vote_zxid) 广播出去, 将收到的票和自己更新后的票放入自己的票箱。若存在相同的选票则直接覆盖。
- 若二者zxid一致, 则比较二者的 id, 若外部投票的id比较大, 则将自己的票中的vote_myid更新为收到的票中的vote_myid并广播出去, 另外将收到的票及自己更新后的票放入自己的票箱。

⑦ 统计选票

- 若已经有过半服务器认可了自己的投票则终止投票, 否则, 继续接收选票。

⑧ 更新服务器状态

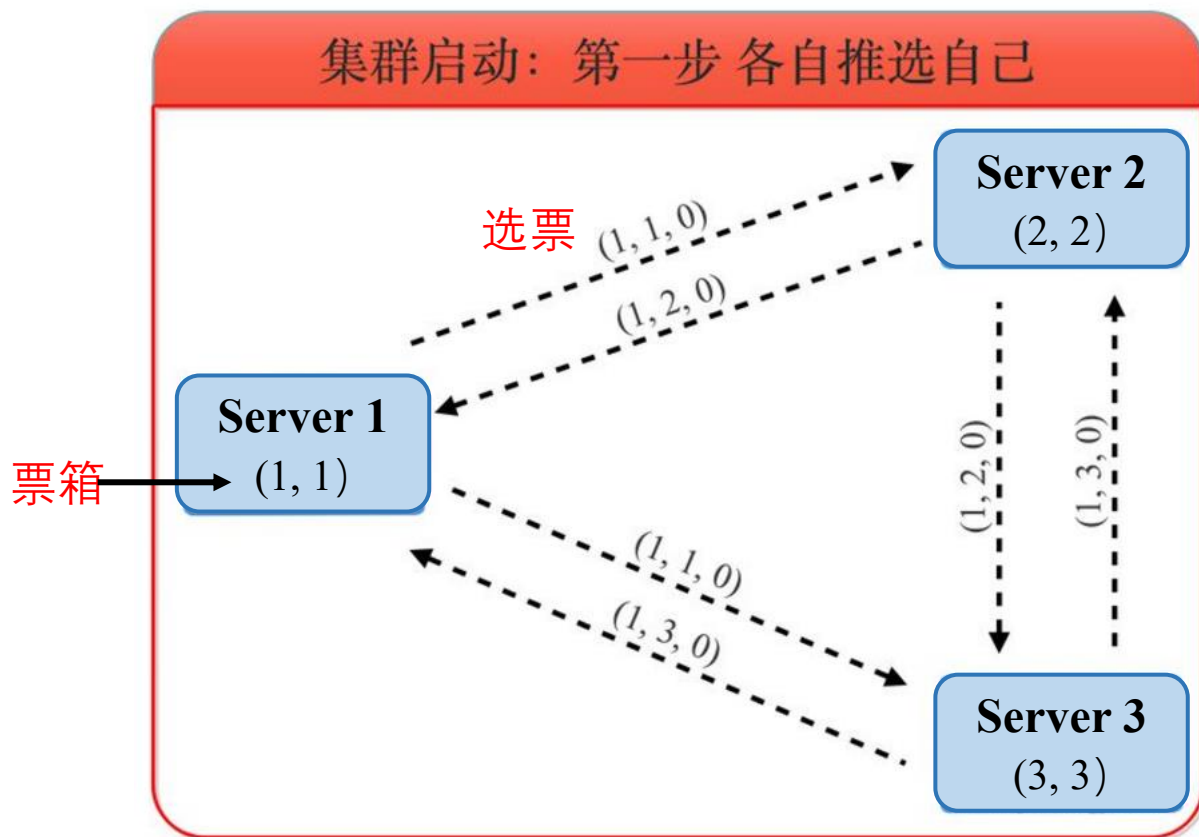
- 投票终止后, 若过半的票投给了自己, 则将服务器状态更新为Leading, 否则将状态更新为Following。

3.3 分布式选举： Zab

◆ 实例1： 集群启动领导选举

□ 初始投票给自己

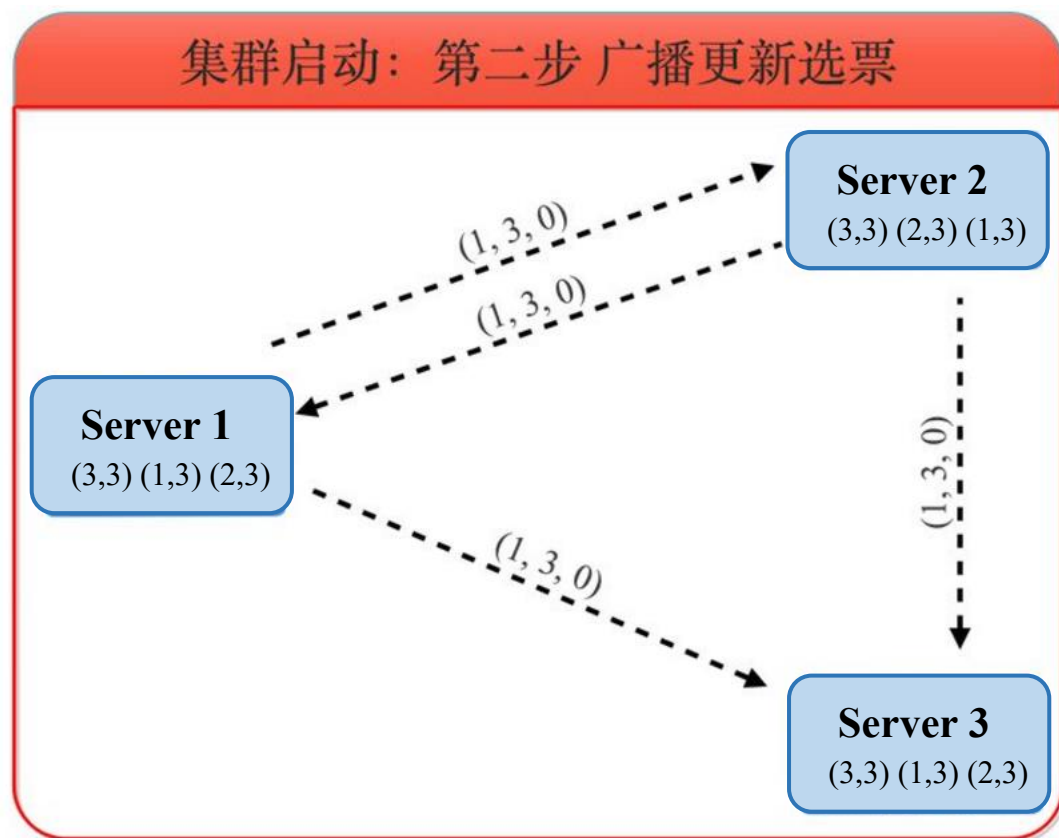
选票 $\langle \text{logicClock}, \text{vote_id}, \text{vote_zxid} \rangle$



3.3 分布式选举： Zab

◆ 实例1： 集群启动领导选举

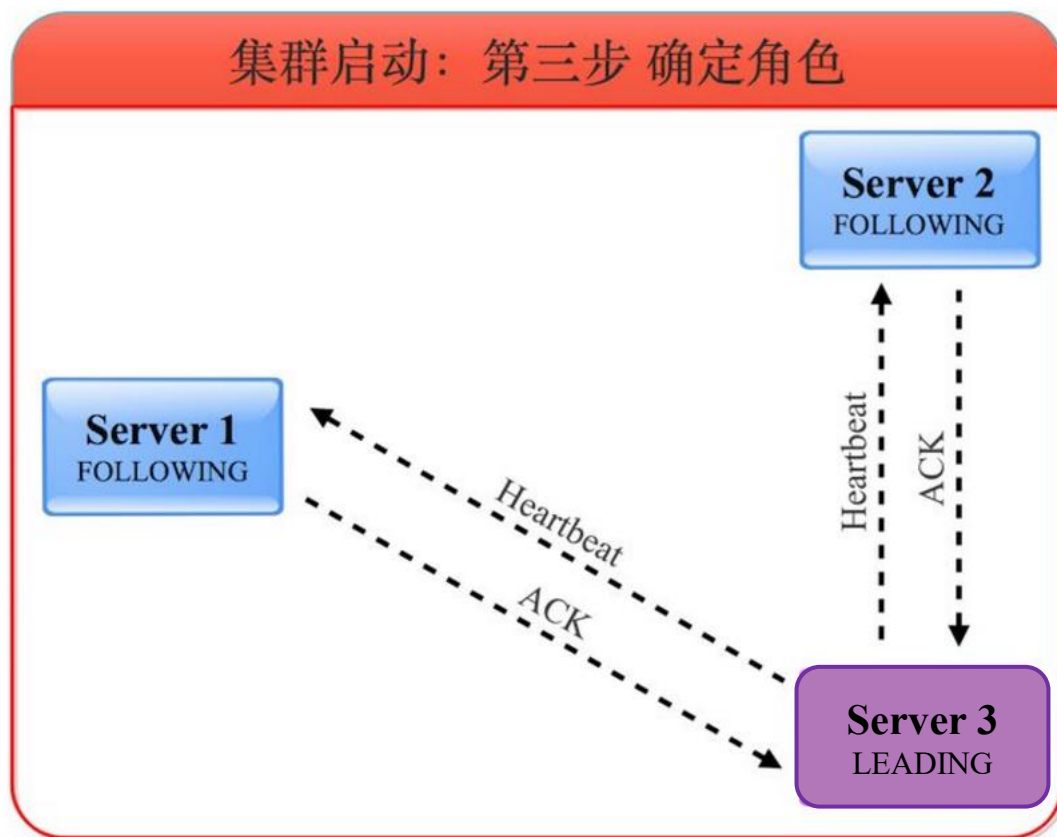
- **更新选票：** 服务器收到外部投票后，进行选票PK，相应更新自己的选票并广播出去，并将合适的选票存入自己的票箱，



3.3 分布式选举： Zab

◆ 实例1： 集群启动领导选举

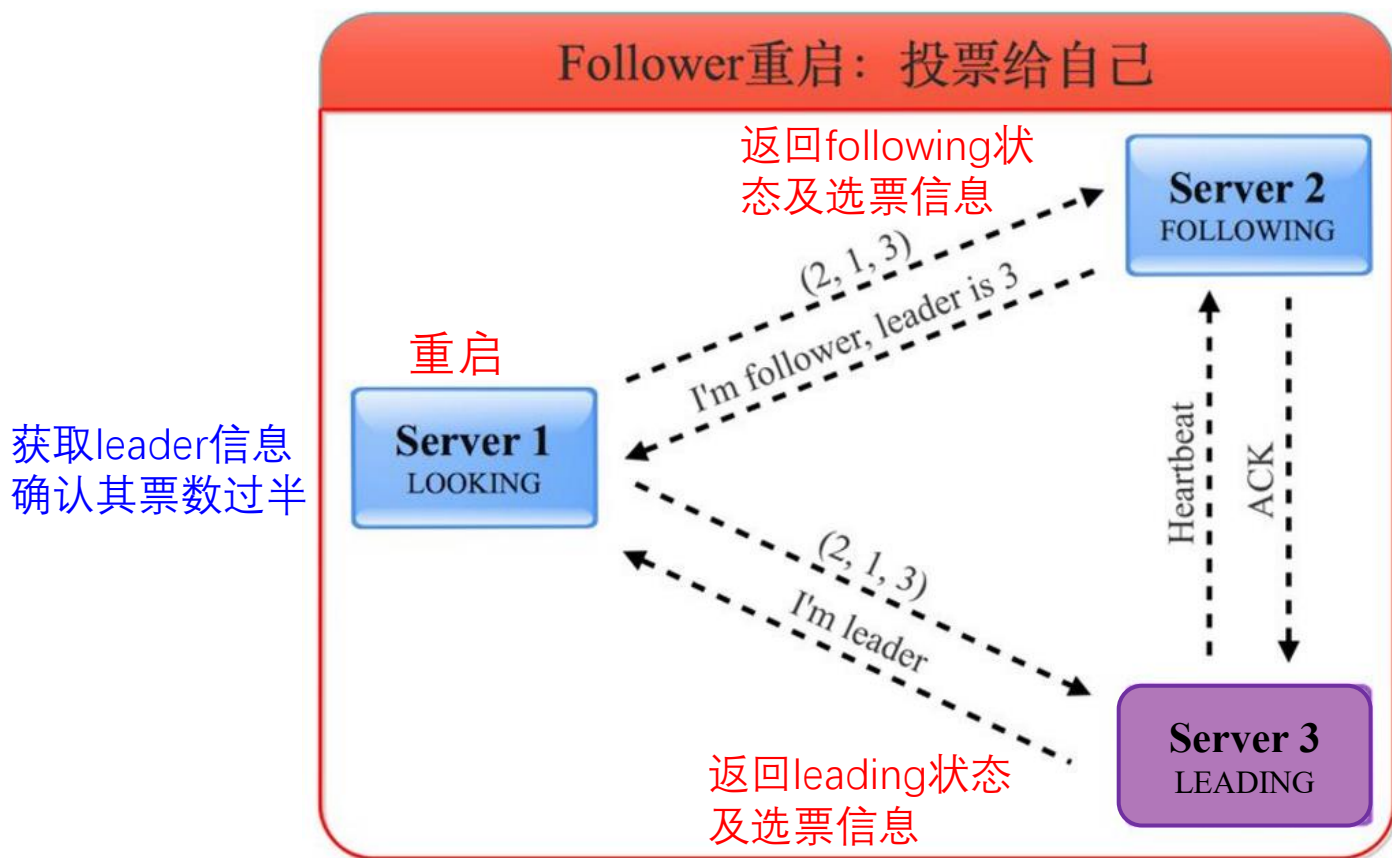
- **根据选票确定角色：** 服务器1和2都进入FOLLOWING状态，而服务器3进入LEADING状态，之后Leader发起并维护与Follower间的心跳。



3.3 分布式选举： Zab

◆ 实例2：Follower重新选举

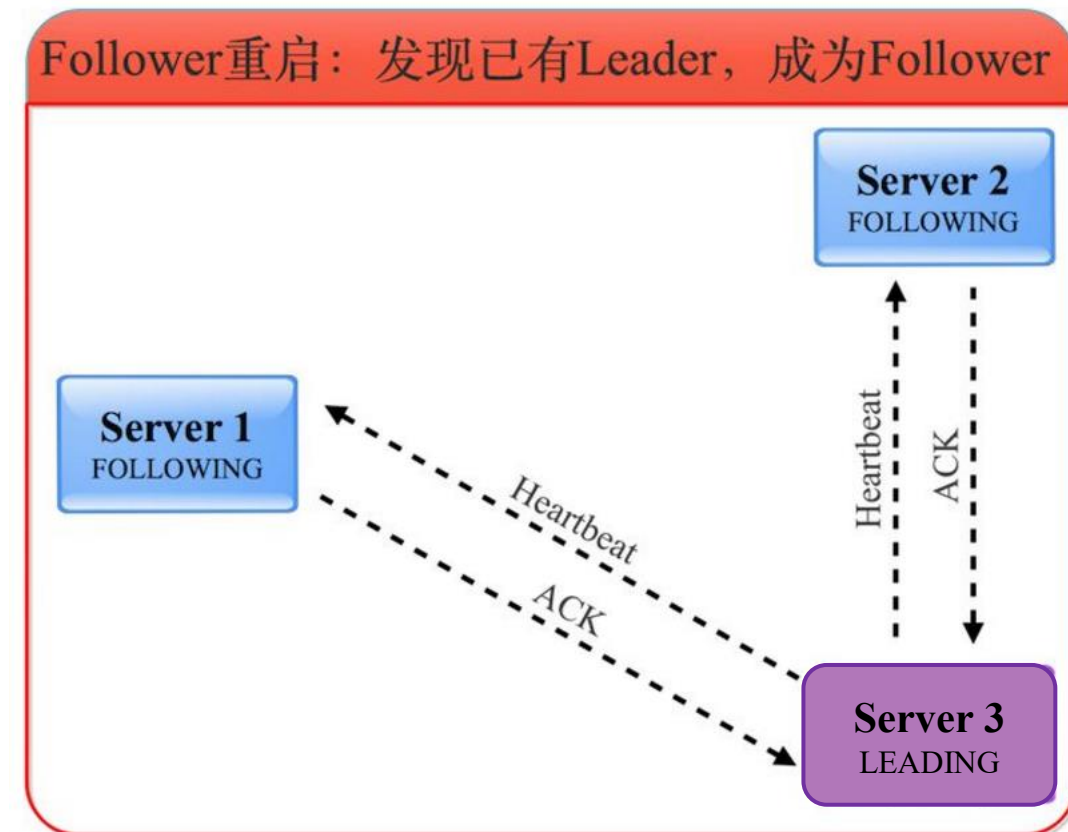
- Follower重启，或者发生网络分区后找不到Leader，会进入LOOKING状态并发起新一轮投票。



3.3 分布式选举： Zab

◆ 实例2：Follower重新选举

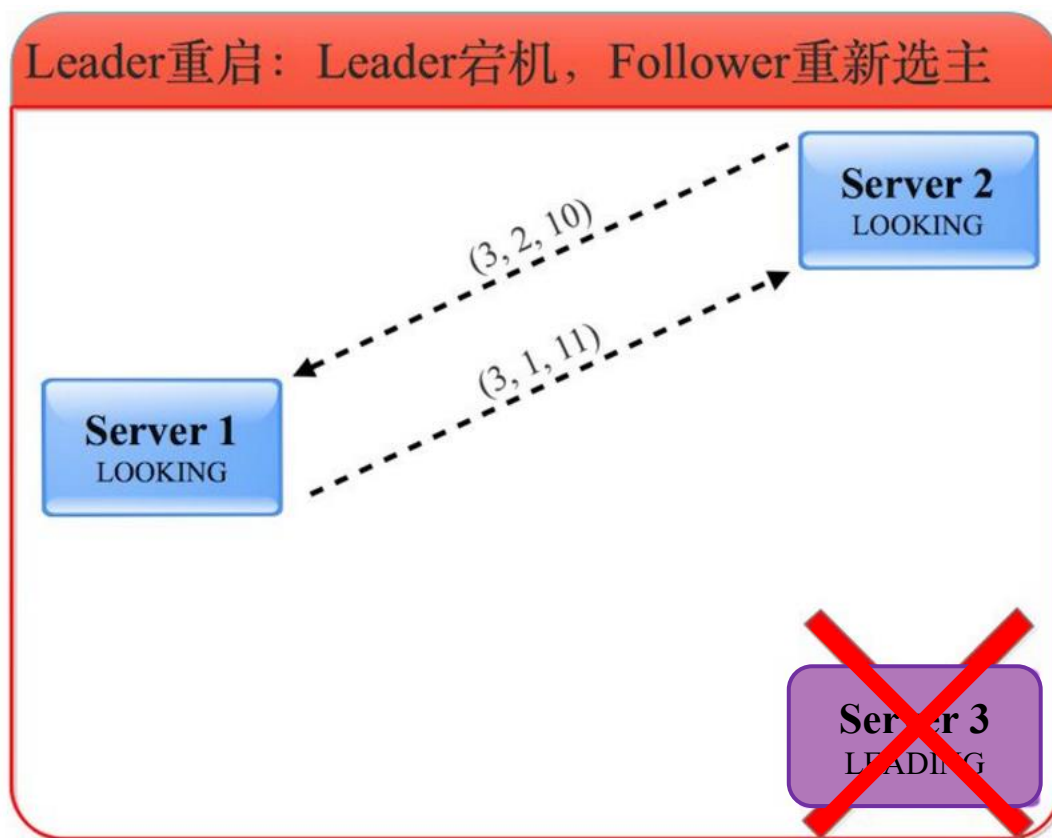
□ 发现已有Leader后成为Follower



3.3 分布式选举： Zab

◆ 实例3：Leader重新选举

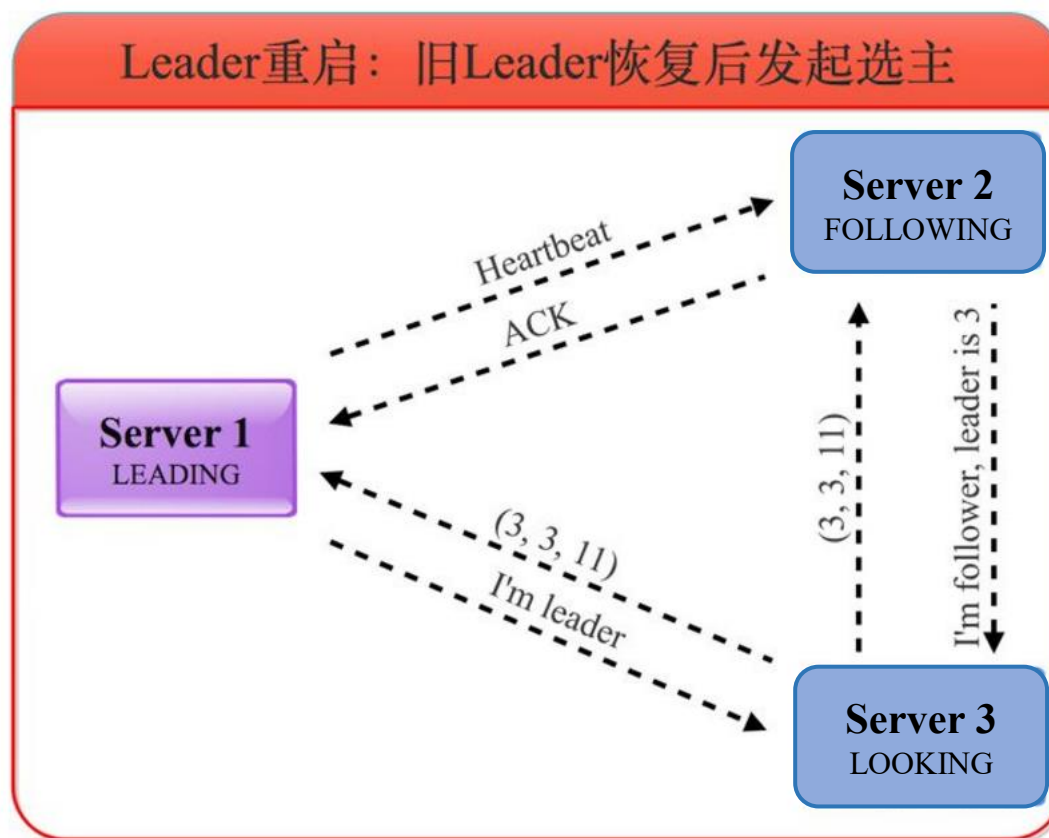
- Leader（服务器3）宕机后，Follower发现Leader不工作了，因此进入LOOKING状态并发起新一轮投票，并且都将票投给自己。



3.3 分布式选举： Zab

◆ 实例3：Leader重新选举

- server1成为新leader，并维护与server2的心跳

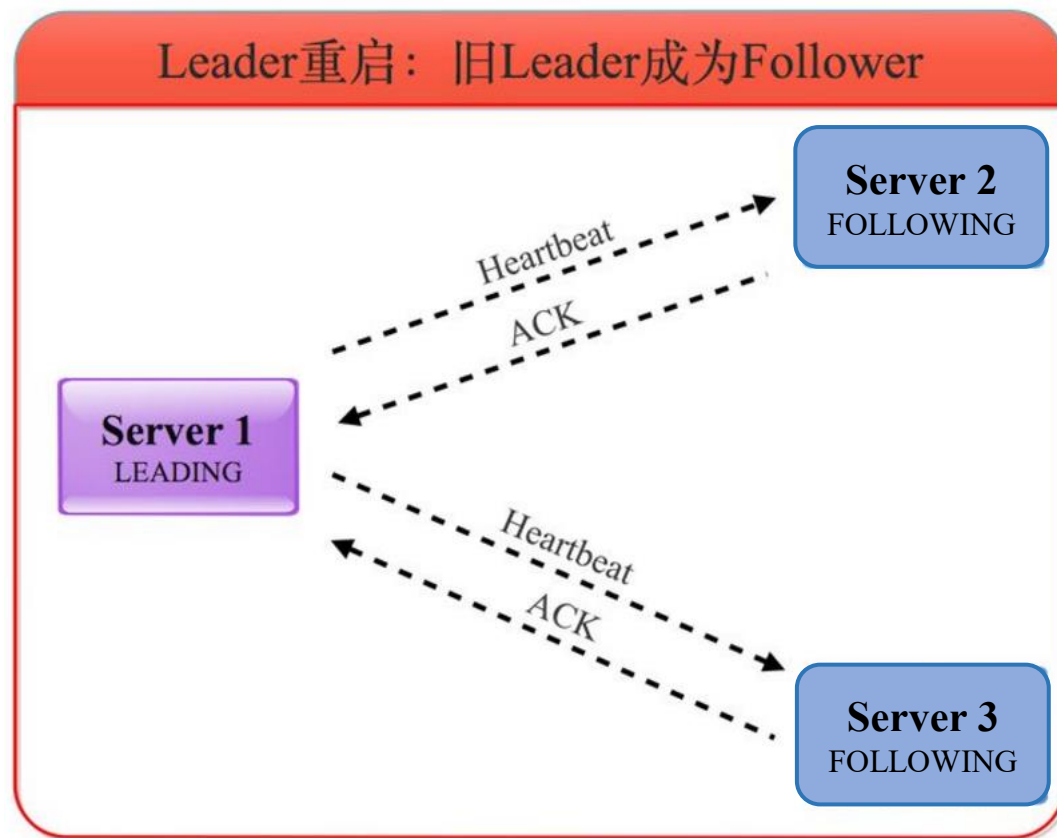


获取新leader信息
确认其票数过半

3.3 分布式选举： Zab

◆ 实例3：Leader重新选举

- 旧leader成为follower

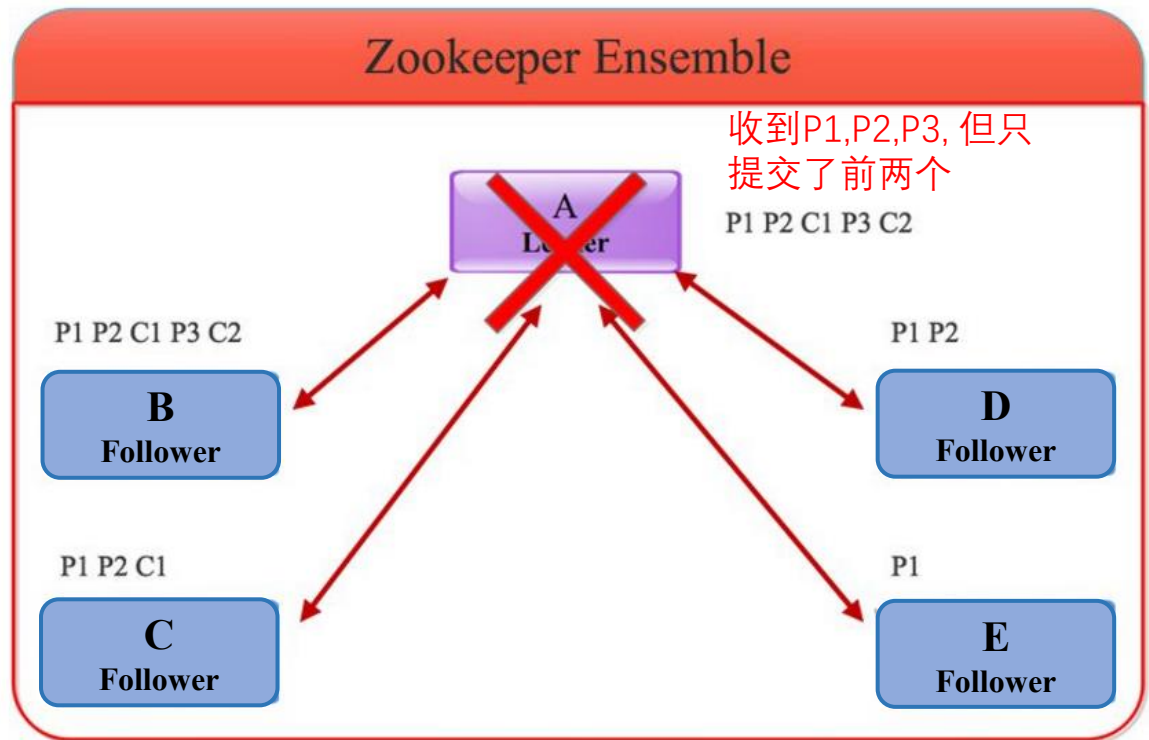


3.3 分布式选举： Zab

◆ 实例4： commit后的数据不丢失

□ failover前的状态

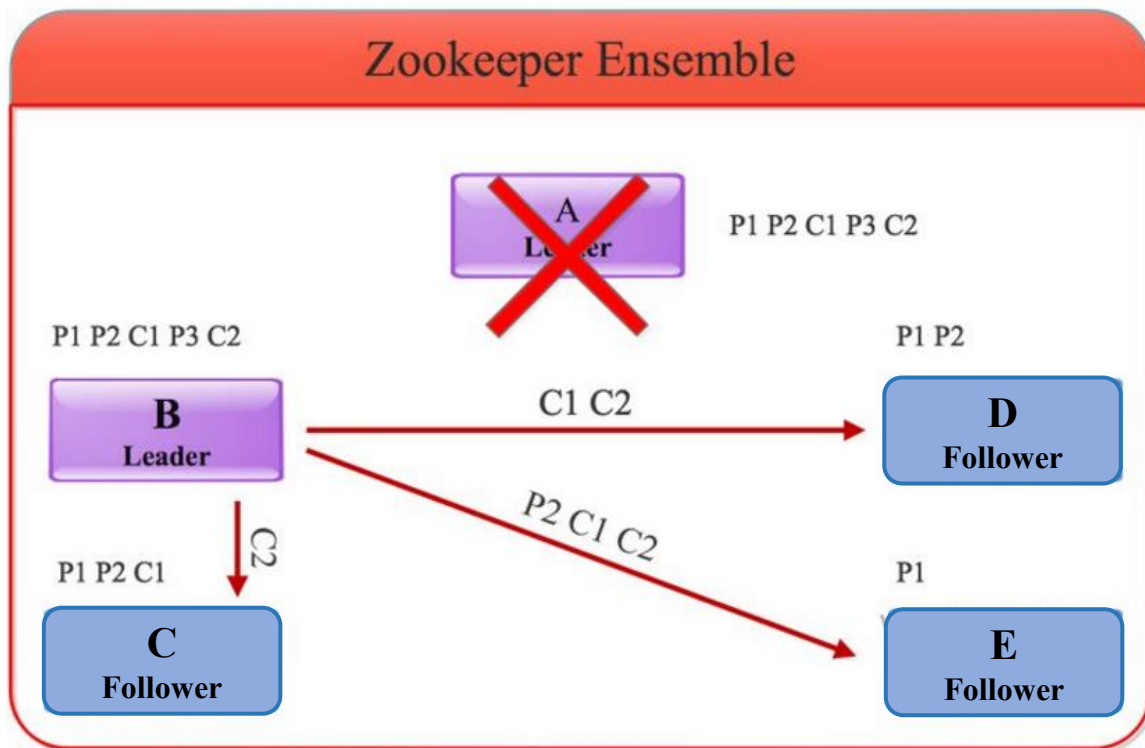
A没有C3，表示收到P3
的服务器总数未过半



3.3 分布式选举： Zab

◆ 实例4：commit后的数据不丢失

- **B被选为新leader**：CDE将自己最大的zxid发送给B，B会将Follower的zxid与自身zxid间的所有被Commit过的消息同步给Follower。

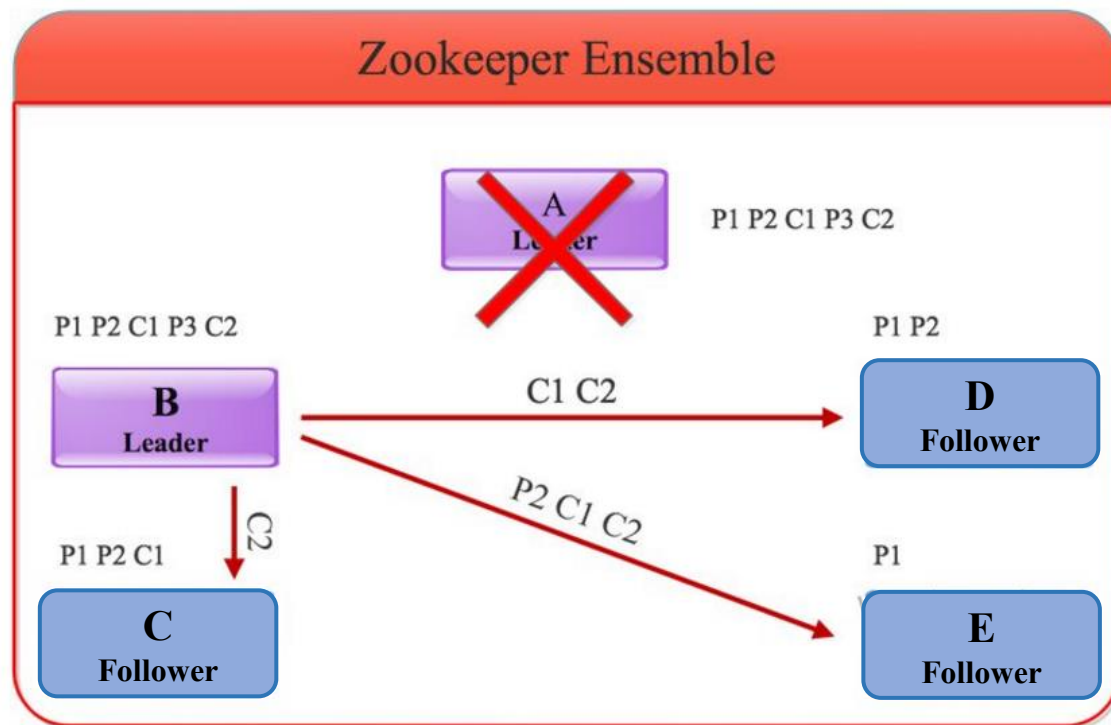


3.3 分布式选举： Zab

◆ 实例4： commit后的数据不丢失

- ▣ **B被选为新leader**： CDE将自己最大的zxid发送给B， B会将Follower的zxid与自身zxid间的所有被Commit过的消息同步给Follower。

- B会通过同步保证P1、 P2、 C1与C2都存在于C、 D和E中
- P3未存在于大多数数据服务器中，因此它不会被同步到其它Follower

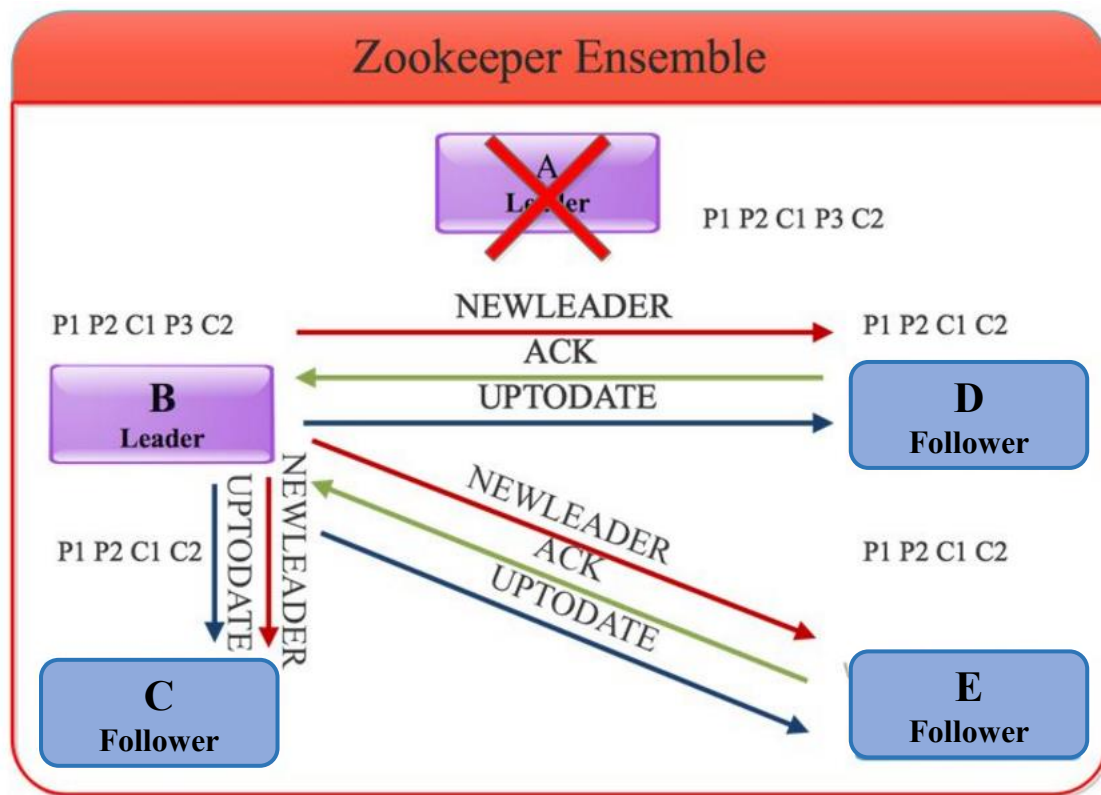


3.3 分布式选举： Zab

◆ 实例4：commit后的数据不丢失

□ 通知follower可对外服务

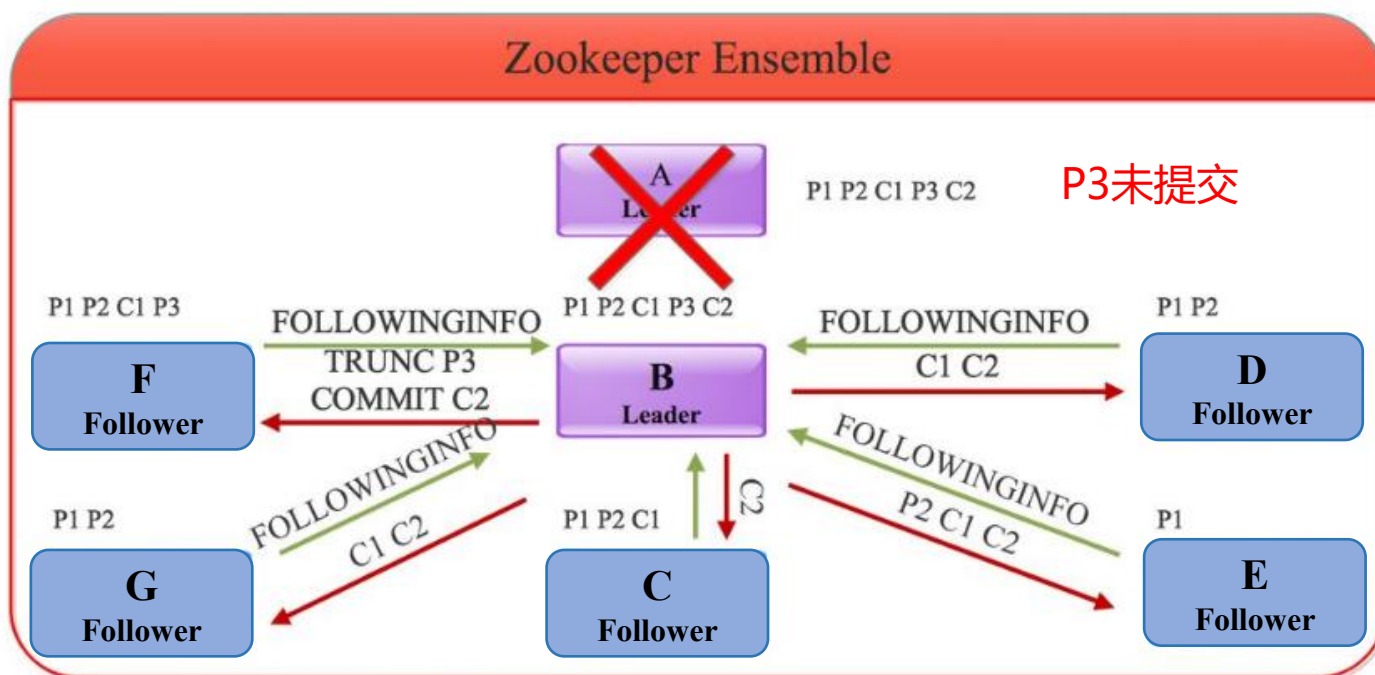
- 同步完数据后，B会向DCE发送NEWLEADER命令并等待大多数服务器的ACK
- B向所有服务器广播UPTODATE命令，收到命令后的服务器即可对外提供服务。



3.3 分布式选举： Zab

◆ 实例5：未commit后的数据对客户端不可见

- 服务器F包含P1、P2、C1、P3，服务器G包含P1、P2。此时服务器F、A和B都包含P3，但是因为票数未过半，因此B作为Leader不会Commit P3，而会通过TRUNC命令通知F删除P3。



3.3 分布式选举： Zab

◆ ZAB选举算法优点

- 算法性能高，对系统无特殊要求
- 稳定性好，不会频繁换主
- 选举过程保证了数据的最新性

◆ ZAB选举算法缺点

- 通信量大：采用广播方式发信
- 选举过程较长：需要同时比对节点ID和数据ID

三种选举算法对比

	Bully算法	Raft算法	ZAB算法
选举消息回复类型	alive消息	同意或不同意选举的消息	投票信息 <epoch, vote_id, vote_zxid>
Leader选举机制	偏向于让ID更大的节点作为Leader	收到过半数的投票，则当选为Leader	倾向于让数据最新或者ID值最大的节点作为Leader
选举过程	只要节点发现Leader无响应时，或者ID较大的节点恢复故障时，就会发起选举	每个角色为Candidate的节点可参与竞选Leader，且每一个Follower只有一次投票权，即同意或者不同意Candidate的选举	每个节点都可以处于Looking状态参与竞选，都可以多次重新投票，根据 epoch、zxid、server_id 来选择最佳的节点作为Leader
选举所需时间	短	较短	较长
性能	Bully < Raft < ZAB		

分布式选举实现算法

Bully

- 选举原则
 - 节点ID最大的称为leader
- 优点
 - 容易理解
 - 选举速度快
 - 算法复杂度低
 - 易于实现
- 缺点
 - 信息存储量大
 - 易频繁切主
 - 不适用于大规模分布式系统
- 应用场景
 - 适用于小规模场景，如MongoDB数据库

Raft

- 选举原则
 - “少数服从多数”，获得投票最多的节点成为主
- 优点
 - 选举速度快
 - 算法复杂度低
 - 易于实现
- 缺点
 - 要求系统全连接
 - 消息通信量大
 - 不适合大规模的集群
- 应用场景
 - 适用于中小的场景，如Kubernetes集群中3个节点的选主方式

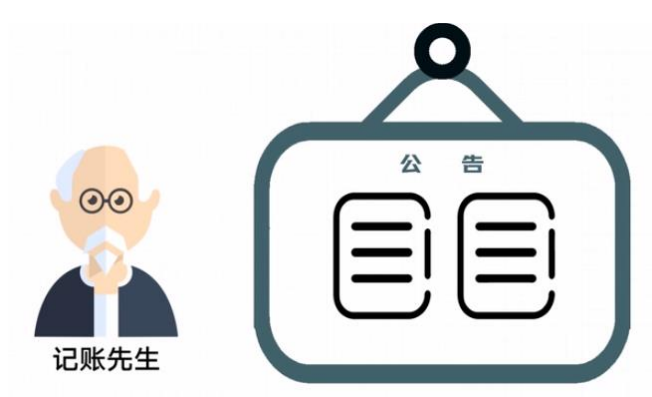
ZAB

- 选举原则
 - 倾向于让数据最新或者ID值最大的节点作为集群的主
- 优点
 - 性能高
 - 对系统无特殊要求
- 缺点
 - 选举时间长
 - 复杂度高
- 应用场景
 - 适用于大规模分布式场景，如Zookeeper

1.3 分布式共识算法

◆ 什么是区块链？

- 区块链=区块+链, 本质是去中心化的分布式数据存储（分布式记账）
- 桃源村的记账故事：无实体货币，一切交易靠大家记账
 - 老王向小明买了100元粮食，广播给村民，大家记账；
 - 村民觉得自己记账麻烦，于是找了账房先生记账，每天5点发布在公告栏。



每天审核，定点发布



村民查看、抄写、加日期、保存

1.3 分布式共识算法

◆ 什么是区块链?

□ 区块链=区块+链, 本质是去中心化的分布式数据存储 (分布式记账)

□ 桃源村的记账故事: 无实体货币, 一切交易靠大家记账

➢ 记账先生每天做结算, 得到每个人的账户余额, 然后添加到第二天账本的开头。

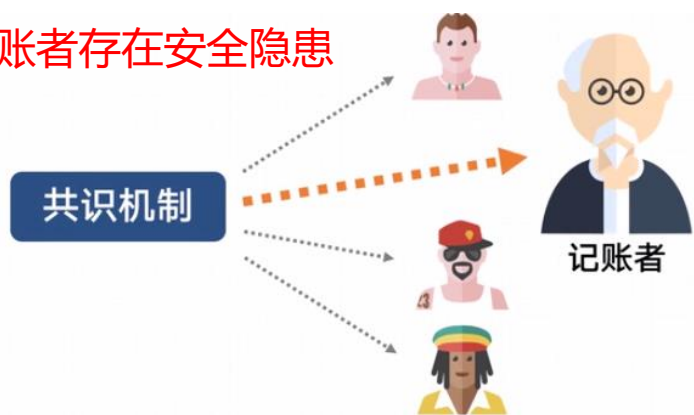


1.3 分布式共识算法

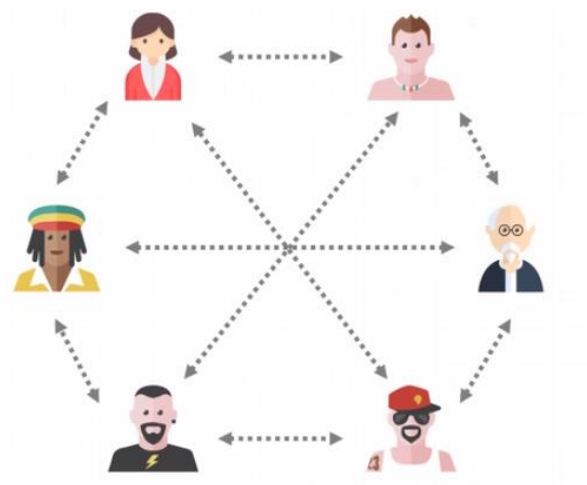
◆ 什么是区块链?

- 区块链=区块+链, 本质是去中心化的分布式数据存储 (分布式记账)
- 桃源村的记账故事: 无实体货币, 一切交易靠大家记账

固定的记账者存在安全隐患



大家都想当记账者, 抛硬币决定,
谁先连续3次正面朝上则成为记账者。
(记账者不固定)

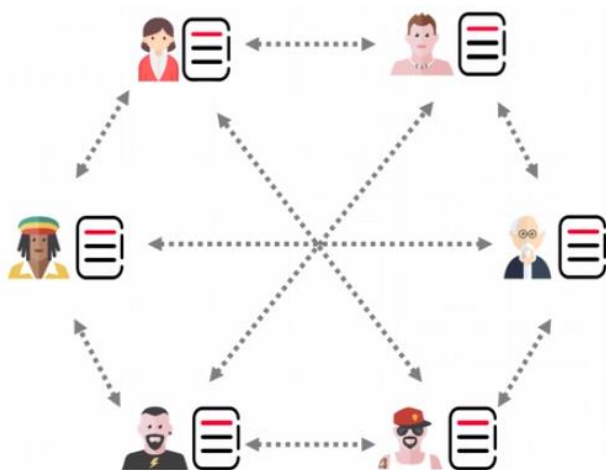


家住的远, 抄邻居的账本
每个人是信息接收者, 也是传递者。

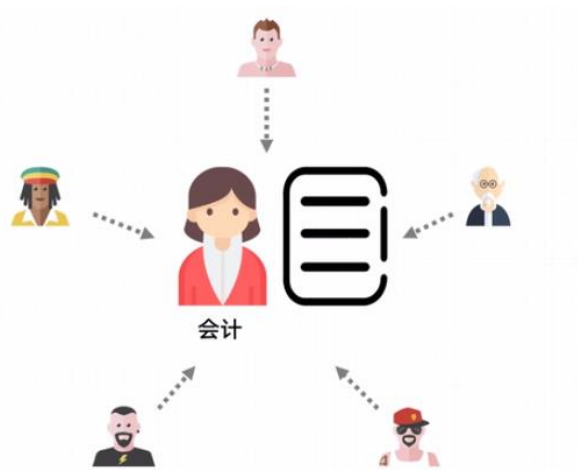
1.3 分布式共识算法

◆ 什么是区块链？

- 区块链=区块+链, 本质是去中心化的分布式数据存储（分布式记账）
- 桃源村的记账故事：无实体货币，一切交易靠大家记账
 - 每个人都有账本，定期检查，淘汰掉与群体不同的账本，如果想要篡改很困难



分布式记账难篡改
(需要攻克全网半数以上的账本)



集中式记账易篡改
(需要攻克一个账本)

1.3 分布式共识算法

◆ 区块链的本质

- 区块链本质：一个人人可见的大账本，只记录交易；
- 核心技术：通过密码学+数据结构，保证账本记录不可篡改；
- 核心功能：创造信任。法定货币依靠政府公信力，比特币依靠技术。

区块链的特征	区块链的支撑技术
P2P网络路由	哈希算法
分布式账本	PoW工作量证明
共识机制	PoS权益证明
难以篡改	公钥/私钥

几个关键问题：

- ✓ 如何分布式记账
- ✓ 如何防篡改
- ✓ 如何激励大家参与记账
- ✓ 如何分配记账权

1.3 分布式共识算法

◆ 问题一：如何分布式记账（利用区块链完成一笔交易）

- 某人发起一笔交易后，向全网广播；
- 每个记账节点，持续监听、传播全网的交易。收到一笔新交易，验证准确性后，将其放入交易池并继续向其他节点传播。
- 每隔10分钟，从所有的记账节点中，按照某种机制抽取1名，将其交易池作为下一个区块，并向全网广播；
- 其他节点根据最新的区块中的交易，删除自己交易池中已经被记录的交易，继续记账，等待下一次被选中。

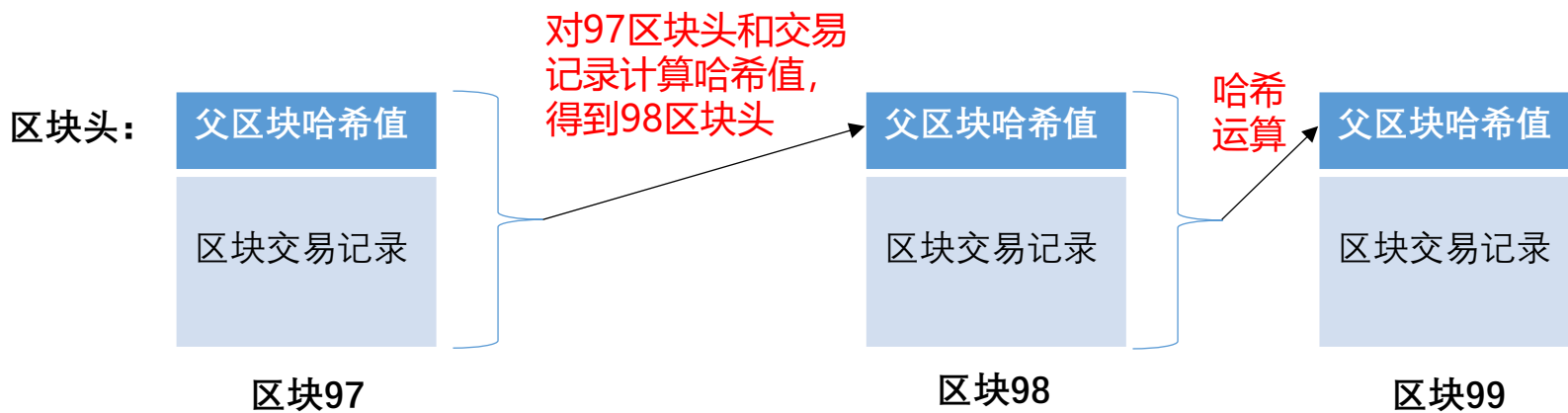
◆ 分布式在线记账

- 任何接入网络的电脑均可参与交易、查看与记录交易
- 每个节点记录的交易信息最终是一致的

1.3 分布式共识算法

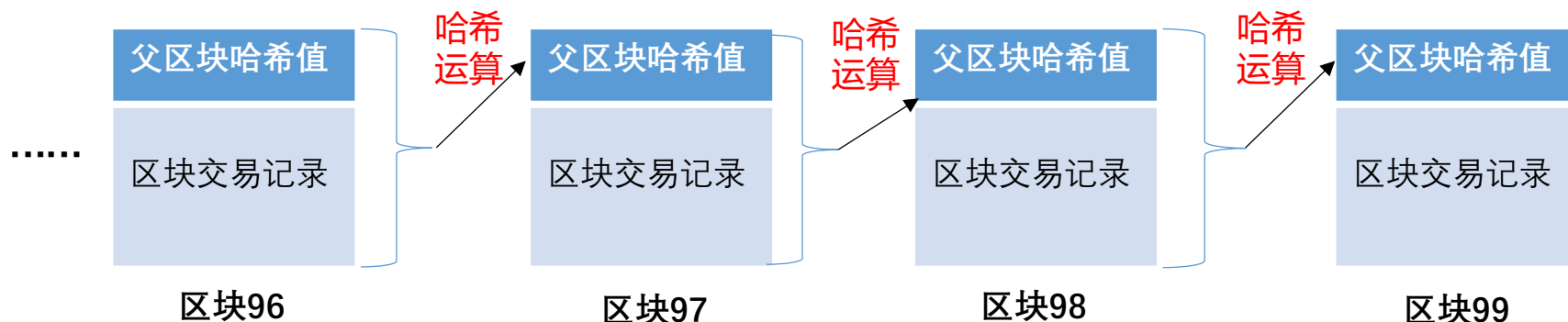
◆ 问题二：区块链如何防篡改？

- 使用哈希函数对交易账本区块进行加密，将区块串成链
 - 将总账本按照区块存储；
 - 每个区块记录一段时间内的交易，例如10分钟；
 - 在每个区块上，增加区块头，其中记录父区块的哈希值；
 - 通过每个区块存储父区块的哈希值，将所有区块按顺序连接起来，形成区块链



1.3 分布式共识算法

◆ 问题二：区块链如何防篡改？



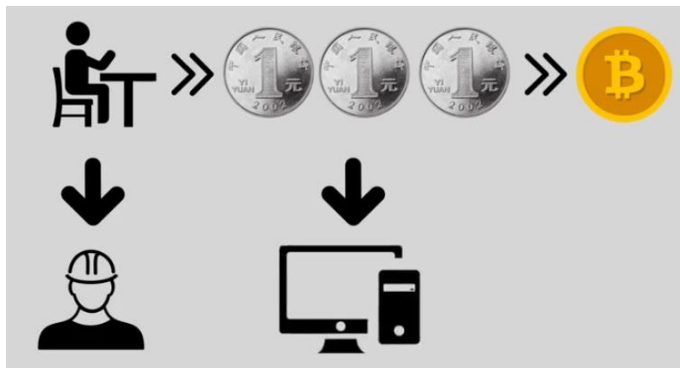
- ❑ 形成区块链以后，篡改任一交易，会导致该交易区块哈希值和其子区块中不同，发现篡改；
- ❑ 即使篡改子区块中的哈希值，会导致子区块哈希值和孙区块中不一致，发现篡改；
- ❑ 只要我们记住最后一个区块的哈希值，任何篡改都能发现。

1.3 分布式共识算法

◆ 问题三：如何激励大家参与记账？

□ 记账者被称为矿工，获得的奖励就是比特币

- 去中心化、全球化、匿名持有、交易记录公开透明；
- 比特币总数量为2100万枚，每四年减产1/2，到2140年，将全部被挖完；
- **具有隐匿性**，在区块链上的账户名只显示一串地址数据，谁也无法知道是谁的账户，而且如果采用比特币转账，追踪不到转账用途。



奖励的方式是获得比特币

竞争的方式：用一定的算力去达成某个条件

1.3 分布式共识算法

◆ 问题四：如何决定记账者，所有节点如何达成一致？

□ 例子：5台服务器分散在北京、上海、华盛顿、伦敦和巴黎，分别对应用户ABCDE，假设用户A给用户B转了100元。假设由上述5台服务器来分布式记账，如何保证记录的交易是一致的呢？

□ 解决方案： 分布式共识算法

- 工作量证明 (Proof of Work)
- 权益证明 (Power of Stake)
- 委托权益证明 (Delegated Proof of Stake)

1.3 分布式共识算法

◆ 分布式共识方法一：工作量证明（Proof of Work）

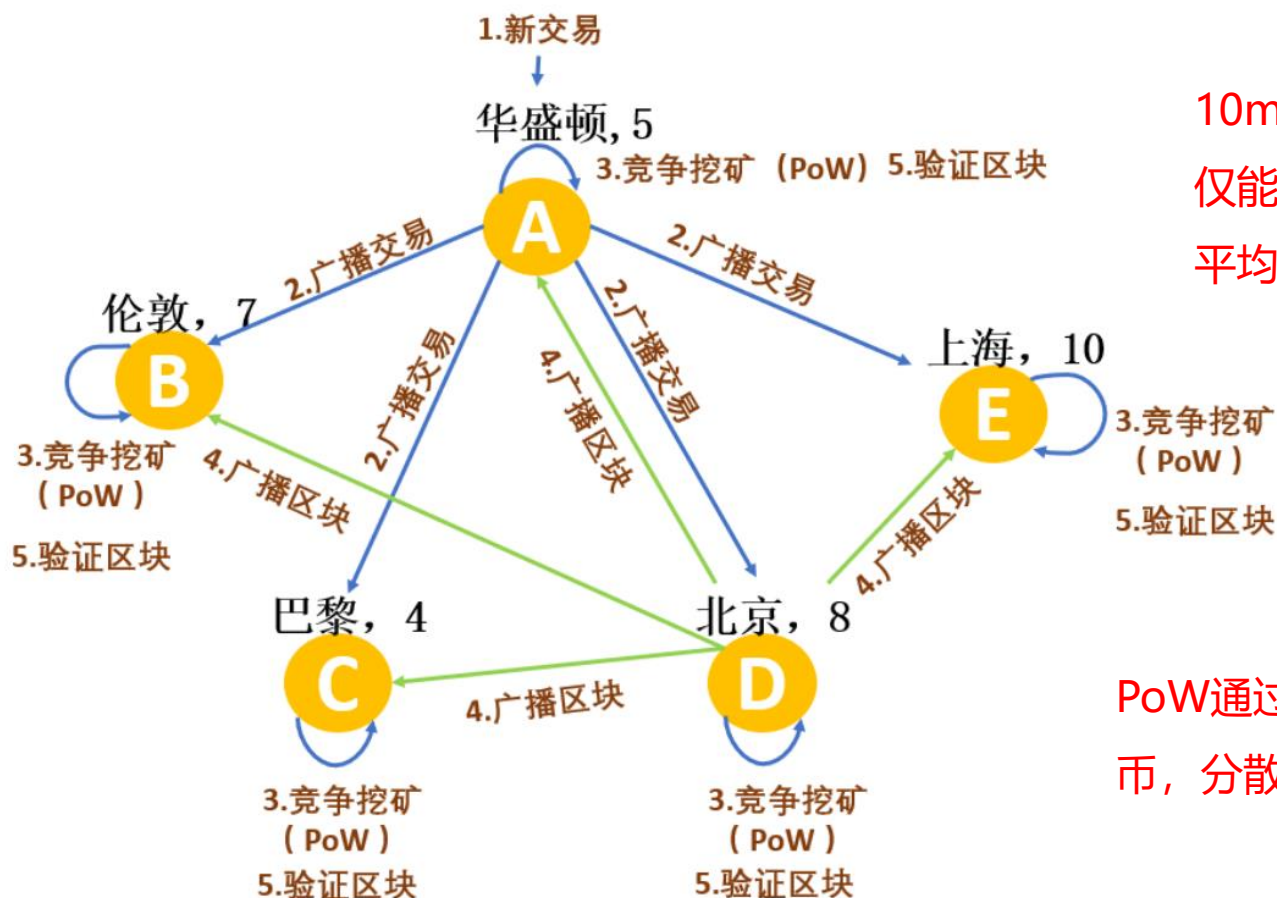
- PoW算法：以每个节点或服务器的算力来竞争记账权的机制。
- 体现算力的方式：通过求解复杂计算题的速度来比较
- PoW算法获取记账权的原理：
 - 利用区块的index、前一个区块的哈希值、交易的时间戳、区块数据和nonce值，通过SHA256哈希算法计算一个哈希值，并判断前k个值是否均为0（小于某数）
 - 如果不是则递增nonce值，重新按照上述方法计算；若是，则本次计算的哈希值为该题的正确答案。谁先算出则获得这个区块的记账权。

注意：nonce是用来找到一个满足哈希值的数字；k为哈希值前缀0的个数；0越多计算难度越大。

1.3 分布式共识算法

◆ 分布式共识方法一：工作量证明（Proof of Work）

▣ 基于PoW的共识记账过程（目前比特币平台采用此法）



10min产生区块大小：1M，
仅能包含3000-4000笔交易
平均每秒只能处理5-7笔交易

PoW通过挖矿的方式发行新
币，分散给个人，相对公平

1.3 分布式共识算法

◆ 分布式共识方法一：工作量证明 (Proof of Work)

- ▣ PoW的容错机制，允许全网50%的节点出错；
- ▣ PoW机制每次达成共识需要全网共同参与计算，增加了每个节点的计算量；
- ▣ 若题目过难，会导致计算时间长，资源消耗多；
- ▣ 若题目过于简单，会导致大量节点同时获得记账权，冲突多，增加达成共识的时间。

PoW机制的缺点：共识达成的周期长、效率低、资源消耗大

1.3 分布式共识算法

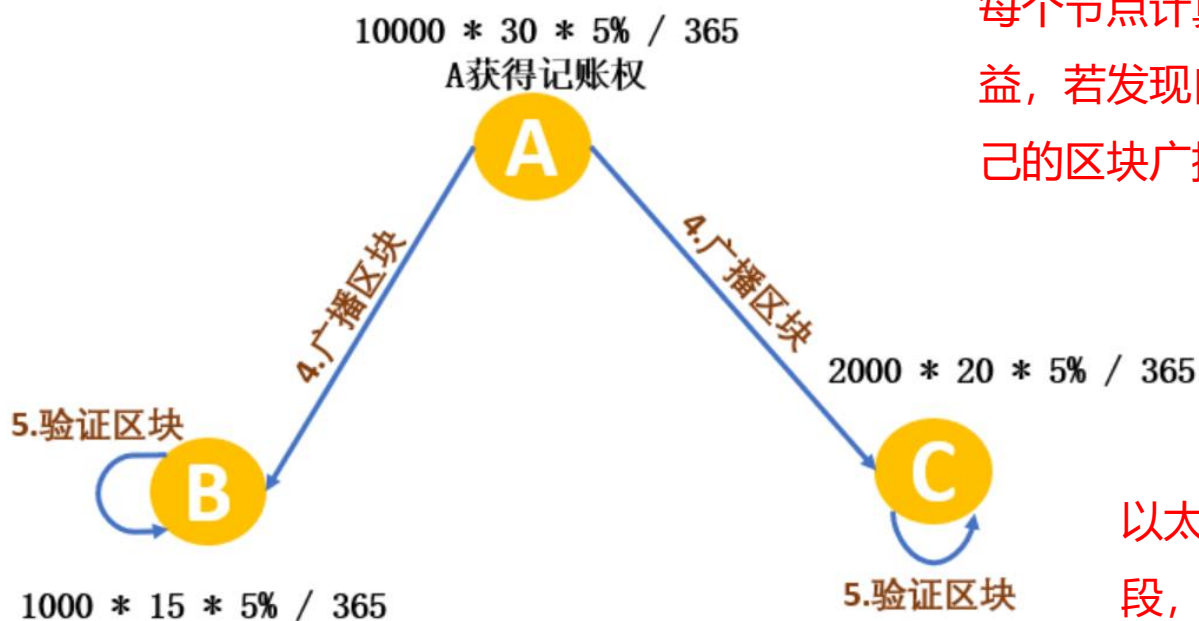
◆ 分布式共识方法二：权益证明 (Power of Stake)

- PoS核心原理：由系统权益代替算力来决定区块记账权，拥有的权益越大获得记账权的概率越大。
- 权益：每个节点占有货币的数量和时间，货币为节点所获得的奖励。
- PoS本质：利用分布式在线记账中的奖励，鼓励“利滚利”。
- PoS模式下，根据持有货币的数量和时间发利息：
 - 持有100个币共50天，币龄为5000。此时发现一个PoS区块，币龄减少365，并获得0.05个币的利息。
 - $\text{利息} = (5000 * 5\%) / 365 = 0.68 \text{个币}$

1.3 分布式共识算法

◆ 分布式共识方法二：权益证明（Power of Stake）

- 案例：网络中有3个节点ABC，其中A拥有10000个币，总共持有30天，而B与C分别有1000和2000个币，分别持有15天和20天。



每个节点计算自己的股权或权益，若发现自己最大，则将自己的区块广播给其他节点。

以太坊平台属于区块链2.0阶段，采用了PoS算法

1.3 分布式共识算法

◆ 分布式共识方法二：权益证明（Power of Stake）

▣ PoS机制的优势：

- 不需要消耗大量的电力就能保证区块链网络的安全性；
- 不需要在每个区块中创建新的货币来激励记账者参与当前网络的运行，缩短了达成共识需要的时间。（基于PoS的以太坊每秒大约能处理30笔交易）

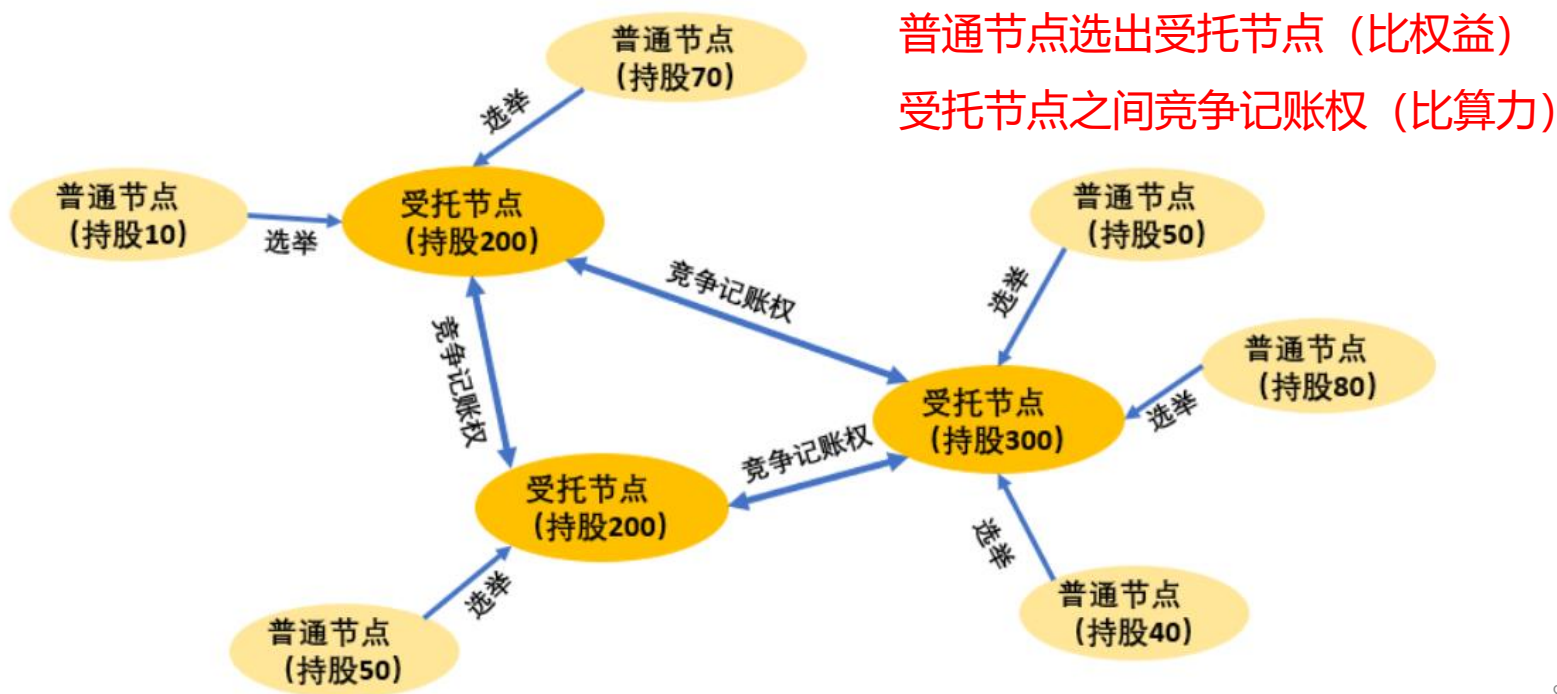
▣ PoS机制的问题：

- 持币越多、越久，币龄越高，就越容易挖到区块并得到激励；
- 整个系统的安全性会被持币数量较大的一部分人掌握，容易出现垄断现象。

1.3 分布式共识算法

◆ 分布式共识方法三：委托权益证明（Delegated Proof of Stake）

- DPoS算法原理：类似股份制公司的董事会制度，普通股民虽然拥有股权，但进不了董事会，他们可以投票选举代表来替他们做决策。
- DPoS是由被社区选举的可信账户（受托人）来拥有记账权，如下图所示：



1.3 分布式共识算法

◆ 分布式共识方法三：委托权益证明 (Delegated Proof of Stake)

- ▣ DPoS在比特股和Steem上运行多年，被称为区块链3.0阶段。
- ▣ DPoS相比于PoS算法，优点表现在：
 - DPoS能耗更低，具有更快的交易速度（只有受托人之间需要算力pk）；
 - 每隔一定周期会调整受托人，避免受托人造假和独权。
- ▣ DPoS存在的不足：
 - 由于大多数持币人通过受托人参与投票，投票的积极性不高；
 - 一旦出现故障点，DPoS无法及时作出应对，导致安全隐患。

1.3 分布式共识算法

◆ 三种分布式共识算法对比分析：

	PoW	PoS	DPoS
计算消耗	高	中	低
结构类型	去中心化	去中心化	去中心化（多中心）
交易量/秒	PoW < PoS < DPoS		
交易服务费	高	低	低
应用区块链平台	比特币	以太坊	比特股



总结

本章我们学习了

- 时钟同步: Lamport算法、向量时钟、矩阵时钟
- 分布式互斥: 集中式算法、分布式协商算法、令牌环算法
- 分布式选举: Bully算法、Raft选举算法、ZAB选举算法
- 分布式共识: PoW、PoS、DPoS